Universidade de Aveiro
Departamento de Eletrónica,
Telecomunicações e Informática
2026

Beatriz Francisco,
Catarina Ribeiro,
Mariana Marques,
Marta Cruz and
Matilde Rodrigues

Group 2

Wearable System for Monitoring Posture and Human Movements

Universidade de Aveiro
2026

Departamento de Eletrónica,
Telecomunicações e Informática

Beatriz Francisco,
Catarina Ribeiro,
Mariana Marques,
Marta Cruz and
Matilde Rodrigues

Group 2

Wearable System for Monitoring Posture and Human Movements

Report on Informatics Project for the Bachelor's Degree in Informatics Engineering at the University of Aveiro, prepared by Beatriz Francisco, Catarina Ribeiro, Mariana Marques, Marta Cruz and Matilde Rodrigues under the supervision of João Almeida, Assistant Professor at the Department of Electronics, Telecommunications and Informatics of the University of Aveiro, Raquel Paradinha and Joana Almeida, PhD Students at IEETA.

Keywords Postural Health, Wearable Sensors, Real-Time Monitoring, Workplace Ergonomics, Three-Dimensional Visualization

Abstract

Maintaining optimal postural health has become one of the most significant challenges in modern professional environments. In both industrial and office workplaces, prolonged exposure to ergonomic risks, such as repetitive movements or static positions, frequently results in chronic musculoskeletal disorders that severely impact long-term quality of life and professional productivity. This project introduces MotionSuit, a wearable system designed for the continuous, real-time monitoring and analysis of human posture. The system is centered on a specialized wearable suit embedded with high-precision sensors capable of capturing body orientation data during natural activity. This information is processed and delivered to a digital platform, where the wearer's physical movements are instantly replicated within a dedicated interface. A core component of this solution is the three-dimensional skeletal visualization, which transforms complex orientation metrics into an intuitive visual model. This allows for the immediate identification of harmful postures, providing users with the necessary feedback to correct their alignment as it occurs. The results of this development demonstrate a responsive and accurate link between physical movement and digital monitoring. By facilitating immediate awareness through real-time feedback, the platform effectively encourages positive behavioral changes in workplace habits. MotionSuit serves as a practical tool for ergonomic evaluation. Its implementation offers a sustainable strategy for the prevention of occupational injuries, significantly enhancing the physical well-being and safety of workers across a wide range of professional sectors.

Contents

List of Figures

List of Tables

Glossary

ACID	Atomicity, Consistency, Isolation, Durability
ADR	Architecture Decision Record
AES	Advanced Encryption Standard
API	Application Programming Interface
CSV	Comma-Separated Values
GDPR	General Data Protection Regulation
GPIO	General Purpose Input/Output
HIPAA	Health Insurance Portability and Accountability Act
HTTP	Hypertext Transfer Protocol
I^2C	Inter-Integrated Circuit
IoT	Internet of Things
IMU	Inertial Measurement Unit
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVP	Minimal Viable Product
PDF	Portable Document Format
PHI	Protected Health Information
PWA	Progressive Web App
RBAC	Role-Based Access Control
REST	Representational State Transfer
REBA	Rapid Entire Body Assessment
ROSA	Rapid Office Strain Assessment
SCL	Serial Clock Line
SDA	Serial Data Line
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTS	Text-to-speech
UML	Unified Modeling Language
WCAG	Web Content Accessibility Guidelines
WLAN	Wireless Local Area Network

CHAPTER 1

Introduction

1.1 Context

Monitoring human movement has become increasingly important for promoting health and efficiency across various daily and professional domains. The rising number of individuals leading sedentary lifestyles, particularly students and office workers, who spend long hours sitting at their desks, often results in poor postural habits and musculoskeletal pain. These habits can lead to chronic conditions, such as lower back pain, and may ultimately become a cause of long-term disability.

By implementing movement monitoring within the workplace (both industrial and organizational), it becomes possible to evaluate occupational safety, production efficiency, and injury prevention more effectively. Furthermore, the ability to accurately track movement allows researchers to optimize workplace design and establish practices that promote long-term health and peak performance.

Conventional ergonomic assessment methods, such as observation and video recording, have significant drawbacks; they are often time-consuming, subjective, and lack the precision required for effective postural analysis. Consequently, the growing demand for objective real-time tracking solutions has driven the development of wearable sensors for the continuous monitoring of human body motion and physiological parameters.

Recent advancements in wearable technology have facilitated the creation of systems that record human motion with high accuracy. Previous research in this field demonstrated the feasibility of integrating diverse sensors, including orientation sensors (BNO055), infrared sensors (MLX90614), and heart rate monitors, into a functional upper-body prototype. The insights gained from this integration and the resulting data processing workflow will serve as the foundation for the upcoming development of a comprehensive full-body monitoring system: the **Motion Suit**. By expanding this technology into a synchronized network of BNO055 orientation sensors alongside physiological modules, the **Motion Suit** provides a holistic, real-time analysis of total body posture and user health. This evolution from a single-limb

prototype to a sophisticated multi-node wearable allows for a more precise and objective assessment of ergonomics in complex work environments.

1.2 Motivation

The increasing prevalence of posture-related health issues has created an urgent need for innovative monitoring solutions, particularly those offering real-time feedback. Current postural assessment methods fail to provide the continuous and objective data required for effective intervention and long-term health management. Furthermore, many traditional wearable monitoring systems do not capture a comprehensive range of motion and posture data, which prevents the accurate measurement of both postural and physiological parameters.

1.3 Objectives

This project aims to design a complete wearable system that will accurately detect and record all full-body human movements in real-time. The device incorporates multiple sensors to gather detailed postural and physiological information from the wearer. The data collected from the sensors will be processed and displayed on a dashboard that allows users to track their posture and movement patterns. To achieve this, users will have access to a web/mobile application that will enable them to review sensor data and receive alerts when their posture becomes potentially harmful. Consequently, this real-time monitoring system is designed for both academic and professional environments to help reduce the incidence of musculoskeletal disorders.

1.4 Document Structure

This document is organized into six main chapters to detail the development and evaluation of the MotionSuit system.

- **Chapter 1 – Introduction:** Provides the context, motivation, and objectives of the project, highlighting the need for real-time postural monitoring in industrial and academic environments.
- **Chapter 2 – State of the Art:** Provides an evaluation of existing technologies, identifying the specific technological gaps that the MotionSuit system addresses.
- **Chapter 3 – Conceptual Modelling:** Describes the theoretical framework and the design requirements for the wearable suit, including the system architecture.
- **Chapter 4 – Implementation:** Details the technical execution of the project, including hardware integration, software architecture, and data pipelines.
- **Chapter 5 – System Analysis:** Presents the legal, ethical (GDPR), risk, perfromance, security and strategic assessment of the ecosystem.
- **Chapter 6 – Results:** Showcases the final full-body suit assembly, latency performance graph analyses, and functional validations.
- **Chapter 7 – Conclusion:** Summarizes main contributions, limitations, and future architectural expansions.

CHAPTER 2

State of the Art

2.1 Related Projects

This chapter provides an examination of available technologies and solutions for monitoring the posture and movement of humans using wearable devices. The analysis will include an overview of the various products available through commercial business entities, along with current research projects that provide solutions to similar issues. An overview of these technologies and solutions is presented below.

DorsaVI

DorsaVI [1] provides a selection of wearable sensor products for various uses in monitoring and analysing human movement. The company makes two product lines aimed at different types of people:

- ViMove+: Targeted at athletes and physical activity, this line enables real-time movement monitoring to enhance sports performance. It generates personalized plans focused on injury prevention and rehabilitation. By analyzing sensor data, ViMove+ assists in creating specialized physiotherapy programs, followed by detailed reports that include corrective exercises to address identified issues. To ensure comprehensive movement capture, sensors are strategically placed on the spine, lower back, shoulders, and arms.
- ViSafe+: This product line is dedicated to occupational health and safety, specifically focusing on the prevention of injuries caused by physical exertion in industrial and organizational environments. By utilizing Injury Data Analytics, the system evaluates historical and real-time data to identify the best safety practices for workers to follow, significantly reducing the risk of workplace accidents. This analytical approach, combined with Advanced Movement Analysis, allows for a precise evaluation of a worker's daily routines to pinpoint potential risk factors and problematic behaviors. Furthermore, the integration of wearable sensors enables active posture intervention through real-time data collection, providing workers with immediate feedback. The information generated is designed to be intuitive and easy to interpret, allowing safety analysts to efficiently

implement corrective exercises and solve ergonomic issues at an early stage, similar to the athletic line, with sensors positioned on the spine, lower back, shoulders, and arms.

FlexiTrace

Unlike sensor-based systems, FlexiTrace [2] utilizes computer vision for postural analysis. The user uploads either a one-minute video or four images captured from front, back, left, and right perspectives into the application for automated postural assessment. Based on these findings, the app generates a customized correction plan tailored to address the user's specific postural issues. To ensure proper execution, the application includes visual guides and videos demonstrating correct exercise techniques. Users are assigned a daily exercise routine, with compliance monitored through streak-tracking functionality. While the reliance on a smartphone camera offers significant accessibility advantages by eliminating the need for specialized equipment, this approach lacks the capability for continuous monitoring provided by dedicated sensor-based systems.

SmartPosture

SmartPosture [3] focuses specifically on monitoring posture during smartphone usage. The application sends immediate alerts via on-screen notifications or vibrations whenever a user adopts poor posture while looking at their device. Additionally, the platform records daily progress to foster long-term habits and a proactive attitude toward maintaining healthy posture during smartphone interaction.

However, due to its specialized nature, the system is only applicable during the active use of a mobile device. It does not utilize external sensors or data capture for monitoring posture during other daily activities. Consequently, SmartPosture does not provide a comprehensive solution for broader postural awareness and feedback unrelated to smart devices, leaving more extensive monitoring needs unaddressed.

SwordHealth

SWORD Health [4] is a digital physical therapy solution that combines wearable devices with artificial intelligence to provide exercise prescriptions for populations undergoing physiotherapy care. The company's proprietary software records user movements during prescribed exercises and immediately analyzes the accuracy of their execution, including specific positions and routines.

To assist the user, the software provides visual demonstrations of the correct performance for each exercise, accompanied by written feedback indicating which aspects require further refinement or correction. At the conclusion of every prescribed therapy session, users receive comprehensive feedback, enabling them to track and measure their progress over time.

While SWORD Health's software ensures continuity between traditional clinical physiotherapy and home-based rehabilitation, its primary focus remains on therapeutic exercise. Consequently, it does not provide ongoing monitoring of a user's posture during everyday activities, leaving a gap in continuous real-time postural analysis for daily life.

VitalJacket

The Vital Jacket [5], developed at the University of Aveiro, represents an early initiative in wearable health monitoring integrated directly into clothing. This garment tracks heart rate, temperature, and other physiological metrics, transmitting the data online for healthcare professionals to review remotely.

In addition to cardiovascular monitoring, the Vital Jacket analyzes sleep disturbances by monitoring nocturnal activity levels and calculating user fatigue. While the Vital Jacket provides an innovative approach to physiological data collection, its primary focus is on vital signs for medical purposes. The Vital Jacket is an innovative new way of monitoring physiological data, but does not focus on posture or how well users perform movements as much as it focuses on monitoring users' vital signs for medical purposes.

Comparative Analysis

Table 2.1 presents a comparative analysis of the reviewed systems against key features relevant to comprehensive movement and posture monitoring, including the proposed MotionSuit system.

Table 2.1: Comparative analysis of posture monitoring systems.

	SmartPosture	**FlexiTrace**	**SwordHealth**	**DorsaVI**	**VitalJacket**	**MotionSuit**
Performance Analysis	Yes	Yes	Yes	Yes	No	+/-
Real-Time Feedback	No	No	Yes	Yes	Yes	Yes
Wearable Sensors	No	No	No	Yes	Yes	Yes
Gamification	No	Yes	No	No	No	+/-
3D Model	No	No	No	No	No	Yes

The analysis demonstrates that while numerous commercial solutions provide movement monitoring or specific aspects of posture assessment, each is characterized by significant limitations. DorsaVI offers rigorous sensor-based monitoring but lacks gamification elements to support long-term user engagement. FlexiTrace provides accessible camera-based analysis; however, this method is incapable of delivering continuous, real-time monitoring. SmartPosture is restricted to a narrow use case involving smartphone usage, while SWORD Health focuses exclusively on therapeutic exercises rather than the ongoing monitoring of posture during daily activities. Finally, the Vital Jacket focuses on physiological parameters for medical purposes but does not address postural or movement analysis.

The proposed system, **Motion Suit**, aims to build upon the strengths of these existing solutions to provide a comprehensive method for monitoring posture and movement throughout the entire day, including work, home, and daily routines. By integrating wearable sensors, analytical data, real-time feedback, and gamification, the Motion Suit offers a holistic approach to health. Additionally, the inclusion of a 3D model for real-time visualization allows users

to better understand and correct their body alignment, filling the gaps identified in current commercial offerings.

CHAPTER 3

Conceptual Modelling

3.1 Problem Statement

The prevalence of sedentary lifestyles, combined with the physically demanding nature of industrial environments, has led to a significant increase in poor postural habits. In factory settings, workers are frequently required to maintain repetitive positions for extended periods, often resulting in incorrect postures and chronic musculoskeletal pain. Despite the high risk of injury in these environments, current methods for evaluating occupational safety and ergonomics remain insufficient. Traditional techniques, such as manual observation and video recording, are time-consuming, subjective, and lack the precision needed to effectively monitor worker health in real-time.

Furthermore, existing technological solutions exhibit notable limitations for industrial applications. While high-end systems like DorsaVI offer specialized tools for workplace safety, they often lack the gamification elements required to foster consistent user engagement. Additionally, many current platforms are restricted to therapeutic contexts or narrow use cases, failing to provide the continuous, full-body monitoring necessary for the dynamic activities of a factory floor.

Consequently, there is an urgent need for an integrated wearable system, such as the proposed Motion Suit, designed specifically to detect and record full-body movements in real-time. By providing objective data, real-time alerts for harmful postures, and engaging 3D visualizations, this system aims to bridge the gap in occupational health monitoring and significantly reduce the incidence of musculoskeletal disorders in both industrial and academic environments.

3.2 System Requirements

The specification process for the MotionSuit can be divided into three main phases. First, we evaluated the system's main business objectives to determine its scope and boundaries, focusing on real-time posture monitoring and biofeedback. Next, we analyzed the state-of-the-art technologies and related projects to identify areas in which innovative features can be

introduced and learn from past experiences. Finally, as a result of brainstorming sessions and consultations with potential users, we gathered the necessary information to be able to create a health monitoring system, involving the following key decisions:

- **Real-time Alerts:** An automated alert system provides immediate feedback to correct harmful movements during repetitive or high-risk tasks.
- **Usability and Accessibility:** The interface must be intuitive and accessible for users with limited technical experience, encouraging consistent long-term use.
- **Advanced Data Analysis:** The platform must support robust filtering and comparison of data across user groups, and enable dataset export for academic and scientific research.
- **Administrative Oversight:** Centralized management is required to oversee user accounts and monitor system performance metrics within defined operational limits.

Context Description

The Motion Suit is designed to adapt to diverse and challenging environments, providing customized solutions for various operational scenarios:

In industrial and construction settings, the system acts as a safety assistant for workers performing repetitive or high-load tasks, requiring accurate movement recognition and low latency despite environmental interference. For students and office workers, the focus shifts to ergonomics during prolonged sitting or standing, with feedback based on established thresholds to promote long-term spinal health. In clinical research contexts, multiple units can be deployed across a participant group to collect synchronized movement patterns, which are uploaded to a centralized platform for collective analysis.

A typical user workflow begins with the calibration of the wearable device to ensure accuracy. When the MotionSuit is fully operational, it continuously records and streams data back to the database, which then processes the collected data to compute the posture score and evaluate it against the REBA [6] or ROSA [7] ergonomic risk thresholds, depending on the assessment method chosen by the user. If an anomaly is detected in the information collected, the individual will be immediately alerted, while the information continues to be saved for future reference.

To ensure an optimized experience within the Motion Suit ecosystem, three primary actors have been defined. Each actor possesses specific access privileges and functional requirements tailored to their role in the system:

- **Suit User:** As the primary end-user, this actor requires an intuitive interface with real-time feedback and personal progress tracking through goals and achievements.
- **Researcher:** Responsible for large-scale data analysis, the Researcher has view-only access to datasets from consenting users and requires tools to filter, compare, export, and identify postural patterns across demographic groups.
- **Administrator:** Tasked with maintaining system integrity, the Administrator manages user accounts, authenticates researcher credentials, and monitors system-level metrics, with elevated access privileges.

Use Cases and User Stories

In this section, the functional specifications of the MotionSuit system are detailed using a complementary approach of User Stories and UML Use Case diagrams. Organized by the three primary actors defined in Section 3.2.3, the User Stories articulate the specific goals and value propositions for the Suit Users, Administrators and Researchers. Complementing these narratives, the Use Case diagrams visually map the structural interactions and workflows required to achieve these desired outcomes. Furthermore, to maintain the security and privacy of the sensitive data collected through MotionSuit, all described use cases and user stories are based on a mandatory authentication layer, restricting access to authorized users only.

Suit User

The Suit User interacts with the Motion Suit ecosystem to receive real-time data and feedback. This information serves as a guide, allowing the user to make conscious postural adjustments and develop healthier habits independently.

Suit User User Stories:

- US-1 **As a** suit user, **I want** to receive alerts about my posture **so that** I can prevent injuries in the future.
- US-2 **As a** suit user, **I want** to have access my complete session history on the app **so that** I can see my improvements over time.
- US-3 **As a** suit user, **I want** to check real-time metrics **so that** I can see my results while I'm working.
- US-4 **As a** suit user, **I want** to customize my alerts **so that** I can adapt the system to my specific needs.
- US-5 **As a** suit user, **I want** to easily calibrate my suit **so that** I can ensure accurate measurements and reliable posture detection.
- US-6 **As a** suit user, **I want** to set personalized posture improvement goals **so that** I can have clear targets to work towards and measure my progress.
- US-7 **As a** suit user, **I want** to visually track my goal progress in real-time **so that** I can stay motivated and see how close I am to achieving my targets.
- US-8 **As a** suit user, **I want** to earn rewards and recognition for achieving my posture goals **so that** I stay motivated and engaged with my posture improvement journey.

The primary functional interactions for this actor are mapped in Figure 3.1, which illustrates the relationship between the user and the core monitoring features.

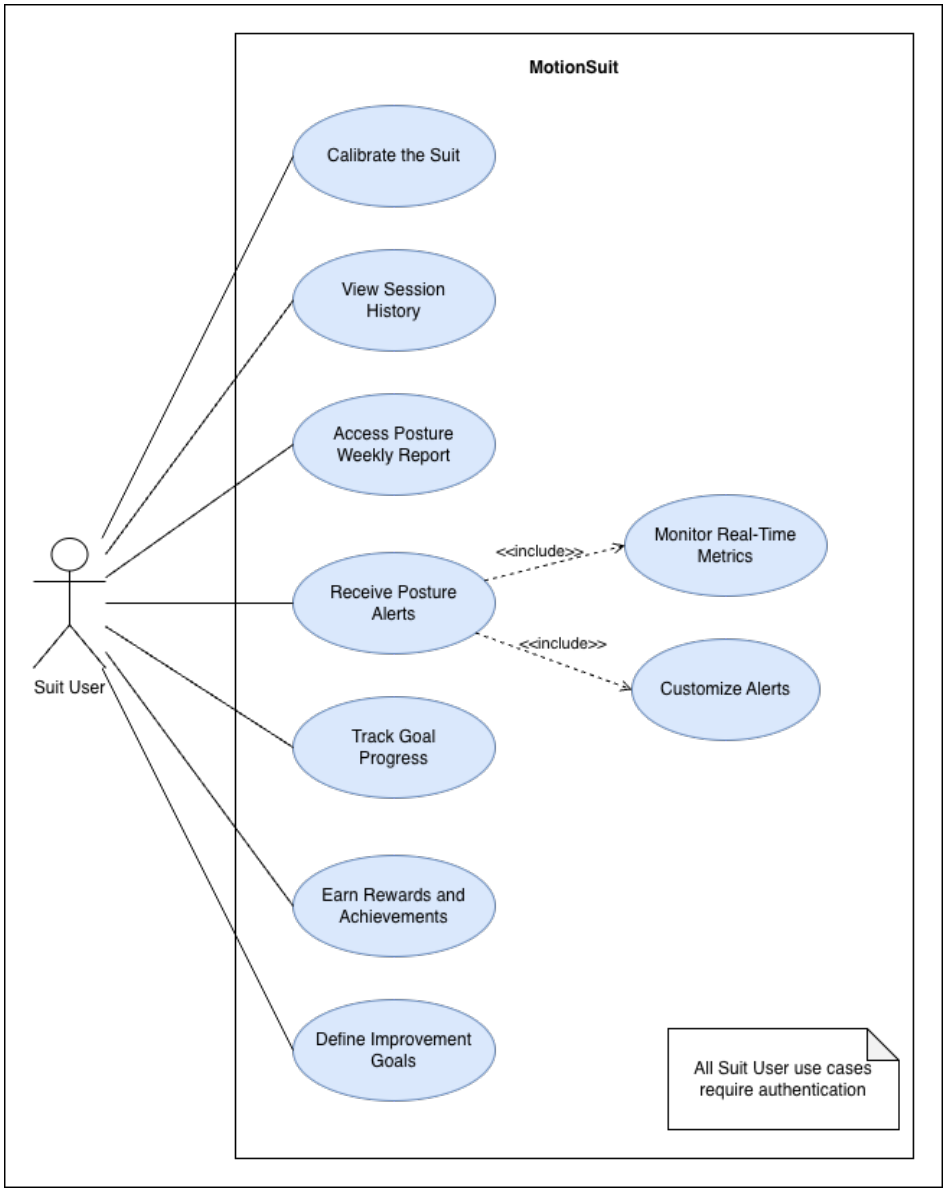


Figure 3.1: Suit User Use Case Diagram.

Suit User Use Cases:.

- **Calibrate the Suit:** The user holds a single reference pose while the system captures sensor readings and computes per-sensor corrections, aligning subsequent movement data to the user's body.
- **View Session History:** Grants access to a complete list of past sessions, allowing users to review their postural history and identify trends and improvements over time.
- **Export Session Data:** Users can export their recorded data in CSV, JSON, or PDF format, selecting sensor data or session history and filtering by date range (last 7, 30, or 90 days, or all time).
- **Receive Posture Alerts:** The system provides real-time feedback when a harmful movement is detected, displaying live REBA or ROSA scores and allowing users to customize notification settings to their preferences.

- **Define Improvement Goals and Track Goal Progress:** Users can establish personalized goals, such as limiting daily slouch duration, with visual progress indicators showing achievement level relative to each target.
- **Earn Rewards and Achievements:** Users are encouraged to maintain healthy habits through achievements earned by preserving daily streaks or meeting specific postural objectives.
- **Access Posture Analytics:** A dedicated analytics page allows users to explore posture data across daily, weekly, monthly, and yearly views, showing average REBA/ROSA scores, session duration, and alert counts, with drill-down into individual sessions to track score variation over time.

Administrator and Researcher

Beyond the primary monitoring interface, the MotionSuit web application facilitates administrative management and scientific inquiry. It allows Administrators to oversee system operations and user accounts as well as their data, while Researchers analyze authorized movement data to identify broader biomechanical patterns.

Administrator User Stories:

- US-9 **As an** administrator, **I want** to manage user accounts **so that** I can control access and provide support.
- US-10 **As an** administrator, **I want** to view real-time system statistics **so that** I can monitor the platform's health and usage.

Researcher User Stories:

- US-11 **As a** researcher, **I want** to analyze the captured data **so that** I can identify postural deviations and abnormal patterns.
- US-12 **As a** researcher, **I want** to compare data between different user groups **so that** I can identify demographic and occupational patterns.
- US-13 **As a** researcher, **I want** to manage experimental studies **so that** I can organize participants into specific study groups.
- US-14 **As a** researcher, **I want** to visually explore the aggregated biomechanical data **so that** I can easily identify trends, correlations, and anomalies.
- US-15 **As a** researcher, **I want** to export specific subsets of captured data **so that** I can download them for external analysis.

The specific administrator and researcher workflows required to support these stories are visually detailed in Figure 3.2.

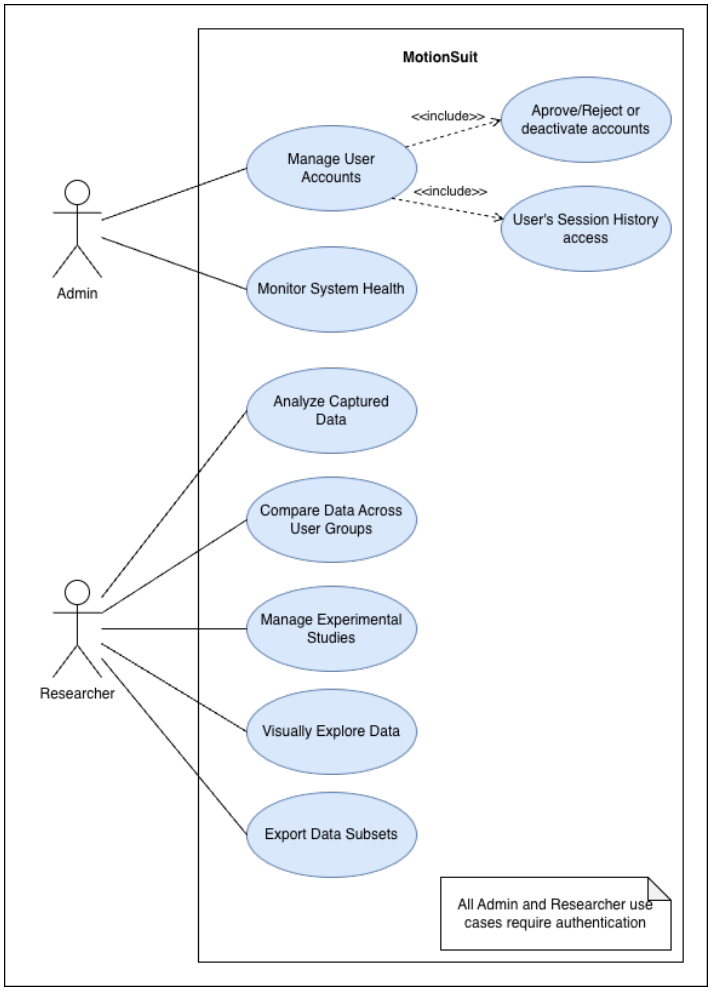


Figure 3.2: Administrator and Researcher Use Case Diagram.

Administrator Use Cases

- **Manage User Accounts:** Allows the administrator to view user details, deactivate accounts, approve or reject researcher registrations, and view per-user statistics such as session count and average duration.
- **Monitor System Health:** Enables the administrator to view real-time system metrics (total users, active users, suit usage), API health, and database status, and to export full database dumps or session logs in CSV, JSON, or PDF format.

Researcher Use Cases

- **Analyze Captured Data:** Researchers can access the movement history of consenting users for longitudinal analysis, enabling detection of postural deviations and abnormal patterns while adhering to privacy preferences.
- **Compare Data Across User Groups:** Researchers can compare postural strain across demographic and occupational groups, such as factory workers versus university students, to identify common risk factors.

- **Manage Experimental Studies:** Researchers can create and manage experimental studies with defined descriptions, timelines, and participant targets, organizing consenting users into dedicated study groups.
- **Visually Explore Data:** Interactive dashboards allow researchers to explore aggregated metrics such as posture scores over time and alert heatmaps, enabling identification of trends and anomalies across the global pool or specific study groups.
- **Export Data Subsets:** Researchers can extract filtered data subsets by timeframe, session, or user group and export them in CSV, JSON, or PDF format for external analysis.

Functional Requirements

The following list describes the functional requirements of the MotionSuit system, that is, the specific behaviors and functions that the system must perform:

- **User Management and Authentication:** The system must support secure registration and authentication for Suit Users, Researchers, and Administrators, with role-based permissions and administrative tools to deactivate accounts and approve or deny researcher requests.
- **Data Capture and Processing:** The system collects raw orientation data from the IMU sensors in real time, computes joint angles for key body segments, and calculates REBA or ROSA ergonomic scores. Historical data is retained for longitudinal posture analysis.
- **Alerts and Notifications System:** The system triggers immediate alerts when a user maintains an incorrect posture for over 15 seconds, delivered via in-app notifications, audio alerts, or visual (beacon) alerts.
- **Visualization and Dashboard:** A real-time dashboard displays live posture data with color-coded indicators (green/yellow/red) and provides historical views through trend graphs filterable by day, week, or month.
- **Goal and Progress Management:** Users can set posture improvement goals, track progress through visual indicators, earn rewards for achievements, and receive automated weekly progress reports.
- **Data Management and Data Export:** Users can download their data in PDF, CSV, or JSON format, and researchers can generate comparison reports across users or time periods for longitudinal analysis.
- **System Administration:** Administrators have access to a dedicated dashboard with system metrics, tools to approve researcher requests or deactivate accounts, and activity logs for accountability.

Non-Functional Requirements

The following list describes the non-functional requirements of the MotionSuit system, that includes the operational and performance limits of the system:

- **Performance and Responsiveness:** The dashboard must load within three seconds and be fully interactive within five seconds for 95% of requests, with biomechanical measurements refreshing within two seconds of acquisition.
- **Security and Privacy:** All health data must be encrypted in transit (TLS 1.3), with credential management delegated to Keycloak and access controlled through role-based permissions (RBAC).
- **Reliability:** The system must guarantee zero data loss during network interruptions through local caching and graceful recovery, support offline functionality, and isolate individual component failures through fault tolerance.
- **Usability and User Experience:** The interface must be learnable by users of diverse technical backgrounds, with a settings page for notification preferences, quiet hours, colorblind mode, text-to-speech, and data sharing consent. The web interface must comply with WCAG 2.1 Level AA guidelines using WAI-ARIA compliant components.
- **Data Management:** User-facing queries must return results within five seconds for 95% of requests. Raw sensor data is retained for 30 days, aggregated metrics for 90 days, and account information permanently unless deletion is requested.
- **Maintainability and Scalability:** All components must expose well-defined APIs documented with OpenAPI, with decoupled backend and frontend containers that can be updated independently, and all architectural decisions documented.

3.3 System Architecture

System Overview

The MotionSuit system implements a secure edge-to-cloud architecture designed to collect, process, and visualize high-frequency biomechanical telemetry. The system is designed around a decoupled edge-to-cloud pattern consisting of an edge processing node (Raspberry Pi) and a unified, modular backend deployed in the cloud.

Security and request orchestration are handled at the network perimeter: all external traffic enters through a reverse proxy acting as an API gateway, while authentication, Single Sign-On (SSO), and Role-Based Access Control (RBAC) are delegated to a Keycloak identity management service.

The structural topology and technological composition of the system are illustrated in Figure 3.3.

The architecture is composed of several discrete components cooperating across the edge and cloud boundaries:

- **Sensors**: Fourteen BNO055 Inertial Measurement Units (IMUs) distributed across skeletal joints to capture real-time physical orientations.
- **Edge Node (Raspberry Pi)**: Interfaced with the sensor array via dual I2C multiplexers. It runs a lightweight Python environment responsible for sensor management, local calibration, data preprocessing, and real-time streaming (see Section 3.3).

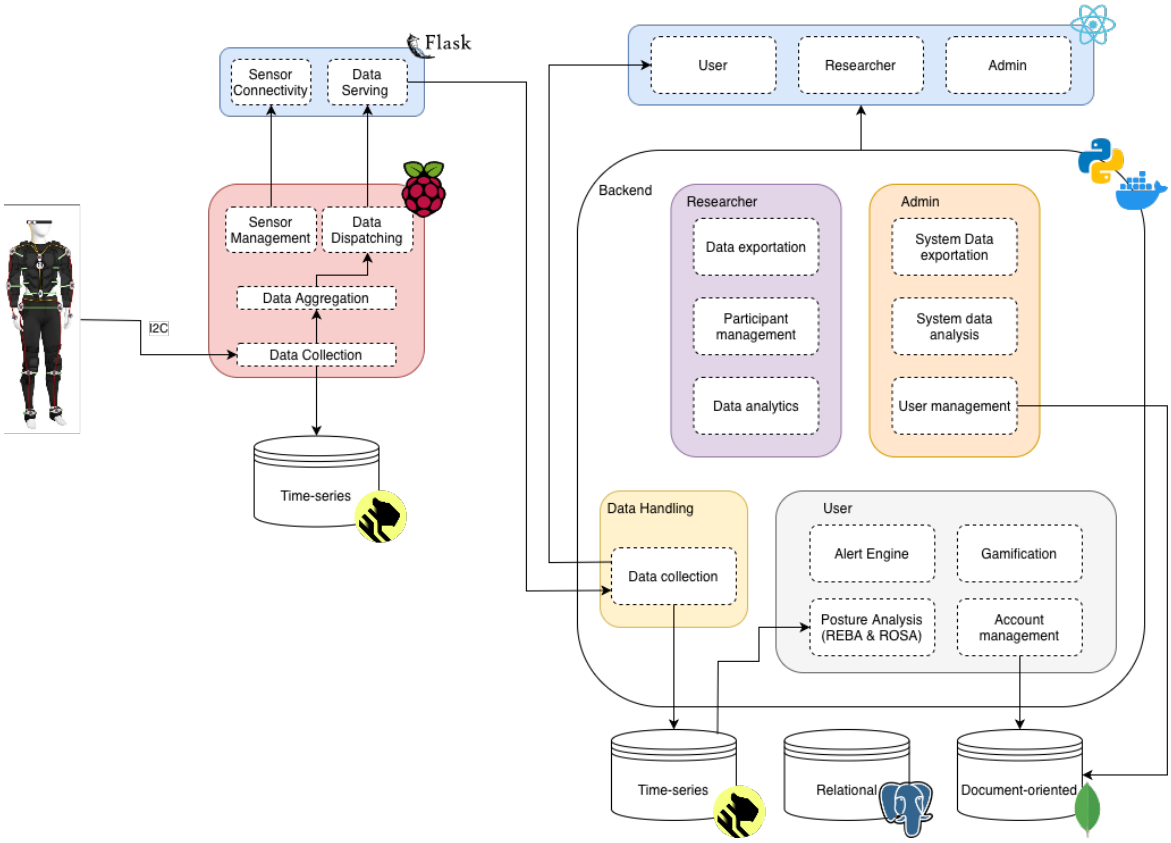


Figure 3.3: System Architecture and Technology Stack.

- **Reverse Proxy and Gateway (Nginx)**: Serves as the unified security gateway at the cloud perimeter, managing SSL/TLS termination and API request routing (see Section 3.3).
- **Identity and Access Provider (Keycloak)**: Secures all resources, managing user credentials, custom onboarding, and role-based access control (RBAC) token generation (see Section 3.3).
- **Application Server (FastAPI)**: The core server engine, structured as a layered application to enforce separation of concerns and handle business logic (see Section 3.3).
- **Storage Tier**: A polyglot storage layout managing relational metadata, central time-series telemetry, calibration baselines, and local edge caching (see Section 3.3).
- **Frontend Views (React / Vite)**: A single-page web application compiled with Vite, providing role-based dashboard interfaces for Suit Users, Researchers, and Administrators (see Section 3.3).

Edge Node (Raspberry Pi) Internal Structure

The edge processing environment running on the wearable suit's Raspberry Pi is composed of several lightweight Python services designed to handle local hardware operations and data transmission:

- **Multiplexed Polling & Sensor Management Service**: Coordinates the discovery, registration, and monitoring of connected sensor hardware. It manages sequential bus

channel switching on the dual TCA9548A multiplexers to poll the default `0x28` address of the 14 active sensors without conflict, and pushes user-specific calibration settings directly to each sensor.

- **Data Preprocessing & Quality Engine**: Interfaces directly with sensor drivers to perform raw data acquisition, filters high-frequency noise, re-maps local coordinate axes, and applies calibration offsets. It conducts strict data validation checks to flag corrupted or implausible readings.
- **Store-and-Forward Caching Agent**: Coordinates circular buffers to manage data backlog during network latency. It aligns timestamps across all sensor streams using hardware clock synchronization protocols and caches processed data in a local time-series SQLite cache on the Pi, guaranteeing data persistence until connection pathways reopen.
- **Telemetry Marshaller & WebSocket Dispatcher**: Packages orientation metrics into compact JSON payloads, applying optimization techniques to minimize packet size. It maintains a secure, persistent, full-duplex WebSocket server to stream real-time telemetry over the secure Tailscale VPN network, supporting heartbeat checks, session authentication, and backpressure handling.

Reverse Proxy and Gateway (Nginx)

The network gateway manages perimeter security and routing for external connections through two main modules:

- **SSL/TLS Termination Module**: Manages certificates and encrypts client connections (HTTPS and secure WebSockets).
- **API Gateway Router**: Evaluates URL paths and proxies incoming requests to backend service containers.

Identity and Access Provider (Keycloak)

Keycloak acts as the centralized identity provider to secure all API resources:

- **User Directory & Realm Config**: Stores logins, credentials, and user profiles.
- **Custom Onboarding & Authentication Flows**: Implements customized themes (using Keycloakify) to manage sign-in, sign-up, password recovery, and email verification.
- **Token Generator**: Issues signed JSON Web Tokens (JWT) containing user identity and RBAC role claims to authorize all requests.

Application Server (FastAPI) Internal Structure

The FastAPI application serves as the central server engine. It is structured using a layered architecture to enforce a strict separation of concerns:

Presentation Layer (Routers/Endpoints)

This layer validates path parameters, handles HTTP requests and WebSocket handshakes, and inspects JWT role claims on every request.

Business Logic Layer (Services)

This layer implements core calculations and holds the business logic for the three system personas. It is partitioned into specialized service modules:

- **Data Ingestion Service**: Handles data acquisition, validation, and sanitization for telemetry data synchronized from the edge node.
- **Alert Engine**: Evaluates live orientations against static posture limits, manages thresholds based on user baselines, and dispatches notifications.
- **Posture Analysis**: Computes aggregate biomechanical statistics, tracks posture trends, and evaluates gamification XP and streak milestones.
- **User Management Service**: Polls Keycloak for user account management.
- **Administration Service**: Coordinates researcher registration approvals, manages user deactivation workflows, and collects operational metrics.
- **Audit Service**: Captures audit logs and operational metrics all across the system.
- **Export Service**: Translates and formats telemetry and study history into PDF, CSV, and JSON files for batch downloads.
- **Sensor Calibration Service**: Collects raw initialization frames and stores baseline reference points for each wearer.

Persistence Layer

Decouples business calculations from data storage systems, using SQLAlchemy models and database object mappers to query and save records.

Storage Tier

The data storage architecture uses a polyglot configuration optimized for high-volume time-series telemetry and structured configuration storage:

- **PostgreSQL (Relational User Metadata)** [8]: Serves as the primary ACID-compliant store for structured account information.
- **TimescaleDB (Central Time-Series Telemetry)** [9]: Serves as the high-performance repository for all historical biomechanical sensor data. It aggregates synchronized data streamed from multiple sessions, executing optimized temporal queries for trend analysis, analytical reporting, and export tasks.
- **MongoDB (Calibration & Baseline Profiles)** [10]: Houses flexible, document-oriented configuration profiles. It stores individualized sensor zero-point reference configurations.
- **Edge Time-Series Cache (Local Pi Storage)**: Implemented on the Raspberry Pi node, this database is responsible for capturing raw measurements, acting as the edge source of truth that preserves telemetry until sync pathways reopen.

Real-Time Data Flow

To support real-time feedback without compromising the security of the application, telemetry data traverses a strict pipeline from the physical sensors to the client browser:

1. **Data Acquisition**: The Raspberry Pi sequentially polls the fourteen BNO055 sensors over the multiplexed I2C buses, reading raw orientation (quaternions) and calibration flags.
2. **Local Processing and Packaging**: The Pi's Edge Service aggregates the raw samples, performs lightweight signal preprocessing (filtering noise), and marshals the synchronized joint coordinates into a compact JSON telemetry frame.
3. **Secure Transmission**: The telemetry frames are dispatched over a secure mesh VPN (Tailscale) connection to the cloud deployment's public IP address.
4. **Ingestion & Gateway Verification**: The Nginx reverse proxy receives the inbound WebSocket stream, validates the TLS handshake, and proxies the telemetry packets to the backend application container.
5. **Token Claims Inspection**: The FastAPI server validates the incoming WebSocket connection by verifying the Keycloak-issued JWT token. Only connections presenting a token with the required role (`suit_user`) are allowed to establish a data-streaming session.
6. **Dual-Path Distribution**: FastAPI immediately relays the JSON frame via an active WebSocket connection to the authenticated user's React frontend for live visualization, and simultaneously pushes the frame to an ingestion queue for persistence in TimescaleDB.
7. **Exoskeleton Rendering**: The React web client receives the quaternion coordinates, parses them, and passes them to a 3D rendering canvas (implemented using Three.js). The engine applies the joint rotations to a virtual skeletal avatar, replicating the user's movements on the dashboard at the current telemetry frame rate.

Frontend Views

The frontend web client application is compiled as a unified single-page application using React and Vite, with role-based routing (enforced by checking Keycloak tokens) to partition the interface into three specialized views:

Suit User View

The Suit User view is designed to help the wearer monitor their body movements, correct ergonomic issues in real time, and track their long-term posture improvement:

- **Real-Time 3D Exoskeleton**: Leverages Three.js (via React Three Fiber) to render a skeletal model that mirrors the wearer's movements. Joint rotations are color-coded in real time to reflect local joint stress.
- **Ergonomic Risk Gauges**: Displays active REBA (Rapid Entire Body Assessment) and ROSA (Rapid Office Strain Assessment) scores computed frame-by-frame by the backend.
- **Postural Alerts Config**: Allows the user to toggle and customize alert modes (in-app notifications, audible warnings, or physical haptic buzzes on the suit) that fire when a bad posture is sustained for more than 15 seconds.

- **Historical Analytics**: Integrates trend charts and filters (daily, weekly, monthly, yearly views) tracking posture scores, alert counts, and suit wearing time.
- **Gamification Board**: Features XP progression tracking, user level badges, daily streaks, and weekly challenges to incentivize healthy working habits.
- **JARVIS Assistant**: A conversational AI interface running on local LLM infrastructures (Ollama), enabling natural language queries regarding posture logs, summaries, and physical recommendations.

Researcher View

The Researcher view provides tools to organize scientific studies, register participants, and analyze ergonomic data across cohorts:

- **Protocol Builder**: Enables researchers to establish clinical or workspace studies with specific deadlines and participant targets.
- **Participant Directory**: Displays cohort participants with demographic metadata (age, occupation, gender), using anonymized IDs to maintain privacy.
- **Biomechanical Analysis Workspaces**: Renders aggregated cohort graphs, historical REBA/ROSA statistics, and alert frequencies across different demographics or protocol groups.
- **Bulk Data Exporter**: Generates and downloads comprehensive study telemetry dumps in CSV, JSON, and PDF formats for offline statistical analysis.

Administrator View

The Administrator view manages overall system configuration, moderates access, and monitors server operations:

- **User Moderation Portal**: Displays all registered accounts. Administrators can deactivate profiles, review pending researcher requests, and approve or reject researcher roles.
- **Keycloak User Deletion Flow**: Triggers Keycloak APIs to execute hard deletions of account credentials while initiating soft-deletion tasks in the local database to preserve audit integrity.
- **System Resource Monitor**: Displays real-time operational metrics for all running containers, including CPU utilization, memory allocations, active WebSocket counts, and database connection pool statuses.

CHAPTER 4

Implementation

4.1 Development Environment

The development of the MotionSuit system was centered on a modular and containerized architecture, ensuring a consistent environment across the various subsystems, including the frontend, backend and sensor data management.

The entire system architecture was orchestrated using **Docker**, which provided an isolated environment for the various services such as the REST API, database systems and data simulation services. By utilizing **Docker Compose**, the integration between these diverse environments was simplified.

Backend and API Development

The core business logic and RESTful API were implemented using **FastAPI**, a high-performance Python framework. FastAPI was selected for its asynchronous capabilities and efficiency in handling high-frequency requests. The backend served as the central hub, managing data flow between the user interface and the storage layer.

To accomplish the diverse data requirements of the MotionSuit system, a hybrid database system was adopted:

- **PostgreSQL:** Utilized as the relational database to ensure data integrity for user profiles and account management.
- **TimescaleDB:** Optimized for time-series data, specifically for recording and querying high-frequency sensor metrics.
- **MongoDB:** Implemented as a document-oriented database to store flexible data structures, such as customized postural baselines.

Frontend and User Visualization

The user interface was developed using the **React** and **Next.js** frameworks. The platform was successfully implemented as a Progressive Web App (PWA) to enhance accessibility across mobile devices. This was achieved by integrating a Web App Manifest for installability and

utilizing **Serwist** to manage Service Workers for advanced caching strategies. For real-time data visualization, **Three.js** was integrated to provide 3D skeletal movement feedback to the user.

Project Management

To maintain version control and collaborative development, **GitHub** was used as the central repository. Project organization, task prioritization and milestone tracking were managed through **Jira** and **Notion**. Additionally, a dedicated microsite was built using **Docusaurus** to provide detailed documentation about the project.

4.2 Hardware Architecture and Sensor Integration

The hardware architecture of MotionSuit is designed around a central processing unit and a network of absolute orientation sensors. Unlike the initial Single-Sensor Minimal Viable Product (MVP), which monitored a single limb, the final system was designed to integrate fifteen Bosch BNO055 inertial measurement units (IMUs) to capture full-body movement and posture. Due to hardware availability constraints, fourteen sensors were deployed in the final implementation. The architecture fully supports the addition of the fifteenth sensor without any software changes.

Communication Topology and I2C Multiplexing

The primary interface between the central processor (Raspberry Pi) and the BNO055 sensors is the Inter-Integrated Circuit (I^2C) protocol. I^2C is a synchronous, multi-slave serial bus system that uses two bidirectional lines: Serial Data (SDA) and Serial Clock (SCL).

A key constraint of the BNO055 sensor is that its I^2C slave address is selectable between only two 7-bit values, `0x28` and `0x29`, depending on the logic state of its COM3 pin. Because the full-body suit requires fourteen separate sensors, placing them directly on the Raspberry Pi's single primary I^2C bus would result in address collisions.

To overcome this limitation, the hardware architecture implements I2C multiplexing using two TCA9548A 8-channel multiplexers. The TCA9548A acts as a bidirectional selector switch, routing the primary I2C bus from the Raspberry Pi to one of eight secondary downstream buses. By assigning different base addresses to the two multiplexers, they can sit in parallel on the main I^2C bus. The detailed mapping of the multiplexers, their addresses, channels, and corresponding body sensor placements is outlined in Table 4.1.

Table 4.1: I2C Multiplexer Channel Assignments.

Multiplexer	Address	Channel	Sensor Location
Upper Body	`0x70`	0	Head
		1	Neck
		2	Left Shoulder
		3	Left Elbow
		4	Left Wrist
		5	Right Shoulder
		6	Right Elbow
		7	Right Wrist
Lower Body	`0x71`	0	Lower Back
		1	Left Thigh
		2	Left Knee
		3	Left Foot
		4	Right Thigh
		5	Right Knee
		6	Right Foot

During operation, the Raspberry Pi writes a control byte to the targeted multiplexer (either `0x70` or `0x71`) to enable the desired downstream channel. The Pi then communicates directly with the BNO055 sensor connected to that channel using the default address `0x28`. This architecture allows sequential polling of all fourteen sensors with zero address conflicts.

The physical wiring setup connects the central Raspberry Pi, the two TCA9548A multiplexers, and the fourteen individual BNO055 sensor modules. To coordinate this complex multi-node topology, the system utilizes a structured wiring harness designed to distribute power and signals efficiently:

- **Power Distribution Rails**: To power the fourteen sensors and the two multiplexers, the Raspberry Pi's 3.3V (Logic Power) and GND (Ground) pins are connected to two central sets of distribution pins (acting as two common power rails). All sensor and multiplexer power wires are terminated at these common rails to ensure a stable, common voltage reference across the entire suit.
- **Primary I2C Bus Splitting**: The Raspberry Pi's hardware I^2C controller pins (SDA on GPIO 2, SCL on GPIO 3) are split in parallel to route the main control signals directly to the primary SDA and SCL inputs of both TCA9548A multiplexers in parallel.
- **Sensor Cable Connections**: Each individual sensor's data lines (SDA and SCL) are routed directly to its designated channel on the corresponding TCA9548A multiplexer. Meanwhile, the power lines (V_{in} and GND) of each sensor are connected directly back to the central distribution rails sourced from the Raspberry Pi.

Cable Assembly and Organization

To minimize physical clutter and wire tangles on the wearable suit, the wiring is organized into custom-built composite cables. The suit's topology groups sensors by body limb:

- **Limb-Based Cabling**: The sensors covering the limbs are grouped into single, bundled cables for each body member (such as the Left Arm, Right Arm, Left Leg, and Right Leg). Each limb cable bundles the wiring for three local sensors (e.g., shoulder, elbow, and wrist for arms; thigh, knee, and foot for legs) and shares the power supply lines within the harness to reduce the total count of individual lines returning to the central unit.
- **Central Sensors**: Sensors located along the body's central axis (such as the Neck and Lower Back) are routed directly to the multiplexers and central power rails, as their central locations do not require grouping into limb bundles.

A simple schematic wiring of the multiplexers to the Raspberry Pi and the distribution rails is illustrated in Figure 4.1.

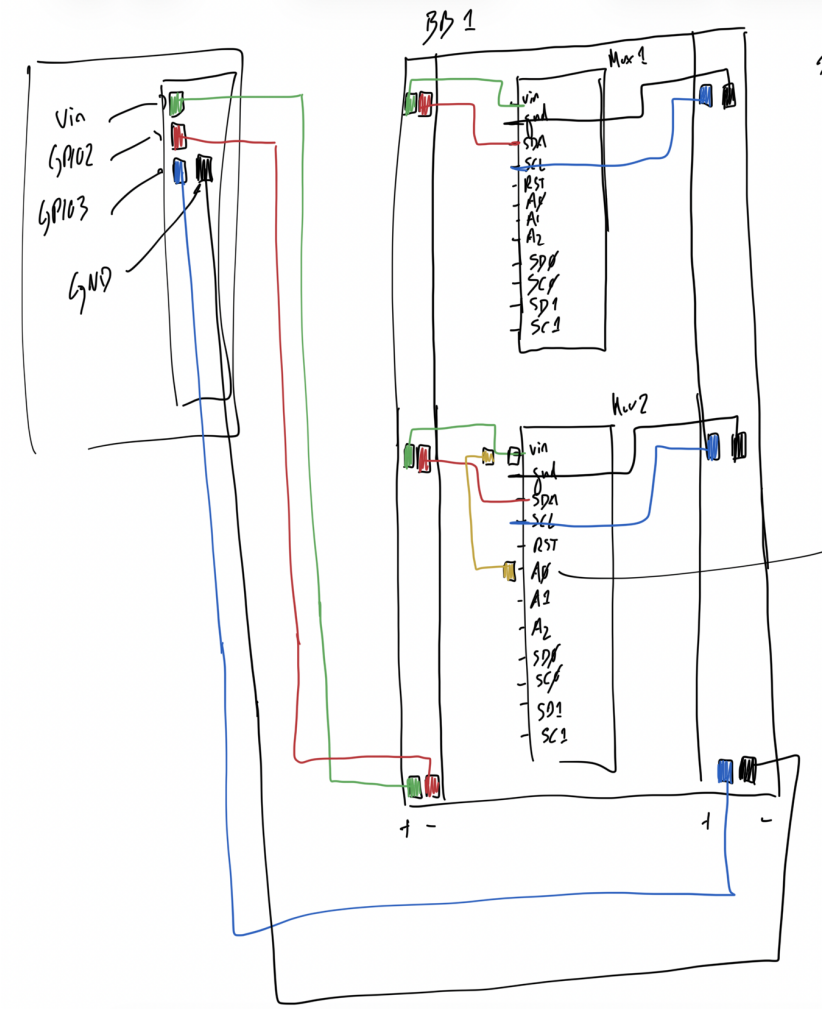


Figure 4.1: I2C Multiplexer and Sensor Wiring Schematic.

The specific pinout of the Raspberry Pi header used to facilitate the primary connections is illustrated in Figure 4.2.

Figure 4.2: Raspberry Pi 40 Pin Header [11].

I2C Bus Configuration and Driver Libraries

To allow the software layer to interact with the physical I2C bus, both the Raspberry Pi's operating system and the Python environment must be configured to support high-speed hardware communication:

- **Kernel Driver Enablement**: By default, the hardware I^2C interface is disabled in Raspberry Pi OS. It is enabled by loading the `i2c-dev` kernel module, which exposes the bus as a device node under Linux (`/dev/i2c-1`). This can be done via the `raspi-config` utility or by adding `dtparam=i2c_arm=on` to the system's boot configuration file (`/boot/config.txt`).
- **Baudrate Optimization**: The default I2C bus baudrate on the Raspberry Pi is 100 kHz (Standard Mode). With fourteen sensors being polled sequentially, the standard speed introduces significant transmission latency, limiting the suit's real-time telemetry framerate. To maximize throughput, the I2C bus speed was increased to 400 kHz (Fast Mode) by adding the parameter `dtparam=i2c_arm_baudrate=400000` to the boot configuration file.
- **CircuitPython Compatibility Layer (`Adafruit Blinka`)**: Since the official Bosch BNO055 driver libraries are written for Adafruit's CircuitPython environment, the system utilizes the `adafruit-blinka` library. This library acts as a translation layer, mapping CircuitPython hardware APIs to the standard Linux `sysfs` and `i2c-dev` kernel interfaces on standard single-board computers like the Raspberry Pi.
- **Bus Access and Device Drivers**: The software instantiates the physical bus object using the standard `board` and `busio` libraries (invoking `board.I2C()`). On top of this bus object, the system layers the `adafruit-circuitpython-tca9548a` multiplexer driver and the `adafruit-circuitpython-bno055` sensor driver to execute register read and write operations.

To ensure modularity and maintainability, the sensor logic is encapsulated within the `RealSensorService` class. This class leverages the `adafruit-circuitpython-bno055` library [12] to interface with the hardware registers, abstracting the complex bit-level manipulations required to configure the IMUs' operating modes. By wrapping these operations, the system decouples the hardware implementation from the application logic, allowing the server to operate agnostically regarding the underlying sensor hardware.

The integration of the BNO055 sensors into the software ecosystem is managed through a dedicated hardware abstraction layer. This layer, implemented in Python, serves as a mediator between the low-level I^2C bus multiplexing operations and the high-level REST API developed in Flask deployed on the Raspberry Pi.

The communication begins with a handshake process where the Raspberry Pi establishes a connection with each BNO055 sensor to verify device readiness. The `RealSensorService` iterates through the configured multiplexer addresses (`0x70` and `0x71`) and their respective channels. For each channel, it attempts to instantiate the BNO055 sensor on the active multiplexer channel. If a sensor fails to respond (for instance, due to wiring noise or connection issues), the service logs the failure, catches the exception, and continues initializing the remaining sensors in the configuration. The service will successfully initialize and start the acquisition thread as long as at least one valid sensor is detected.

Data Acquisition and Performance Optimizations

Data retrieval is performed through a synchronous polling mechanism running in a dedicated background thread (`_update_loop`). The background thread continuously polls all active sensors at a target interval of approximately 30 ms (approximately 33 Hz sampling frequency) and stores the resulting frame array in shared thread-safe memory.

To maximize the sampling frequency and prevent lag in the physical suit, the sensor integration layer implements several critical performance optimizations:

- **Local Euler Angle Calculations**: Reading both quaternions and Euler angles from the BNO055 registers involves multiple slow I^2C read cycles, which severely congests the multiplexed bus. To optimize bandwidth, the background thread reads only the Quaternion registers (reducing the register queries to a single block read). The Euler angles (heading, roll, and pitch) are then computed locally in Python using high-speed trigonometric formulas.
- **Fault-Tolerance Bypass (Throttling)**: A disconnected sensor or transient communication error on the I^2C bus would normally block the polling thread due to bus timeout delays (up to 100 ms per attempt), causing severe latency across the entire suit. The system implements a threshold-based deactivation: if a sensor fails five consecutive times, it is temporarily bypassed (blacklisted) for 5.0 s. This prevents unresponsive hardware from slowing down the active sensors.
- **Pre-Serialization in Background**: To minimize CPU usage on the Flask request handler, the background thread serializes the active frame array into a JSON string (`latest_json`) during the polling cycle. The Flask endpoint (`/api/sensor`) can then return the response immediately without any deep-copying or dynamic serialization overhead in the main web thread.

The final output is structured into a standardized dictionary format containing nested JSON objects matching the schema used by the simulation service, ensuring seamless transitions between real-time sensing and simulated data streams.

The data acquisition layer running on the Raspberry Pi gateway supports two operational modes. In real mode, all fourteen BNO055 sensors are active, capturing live inertial data across the full body, which drives the real-time 3D skeletal visualization on the dashboard. In simulation mode, a dedicated script generates synthetic joint data to emulate natural human movement: a REBA simulation replicates dynamic standing postures such as lifting and carrying loads, while a ROSA simulation replicates seated office behaviour including progressive slouching. The user selects the active ergonomic method before starting a session, and the system switches between modes accordingly.

4.4 Data Processing Pipeline and Communication Protocols

The MotionSuit system implements a structured data processing pipeline designed to efficiently transport high-frequency kinematic telemetry from the wearable device to visual feedback for the user and long-term storage for analysis. The sequential flow of data through this pipeline is illustrated in Figure 4.3.

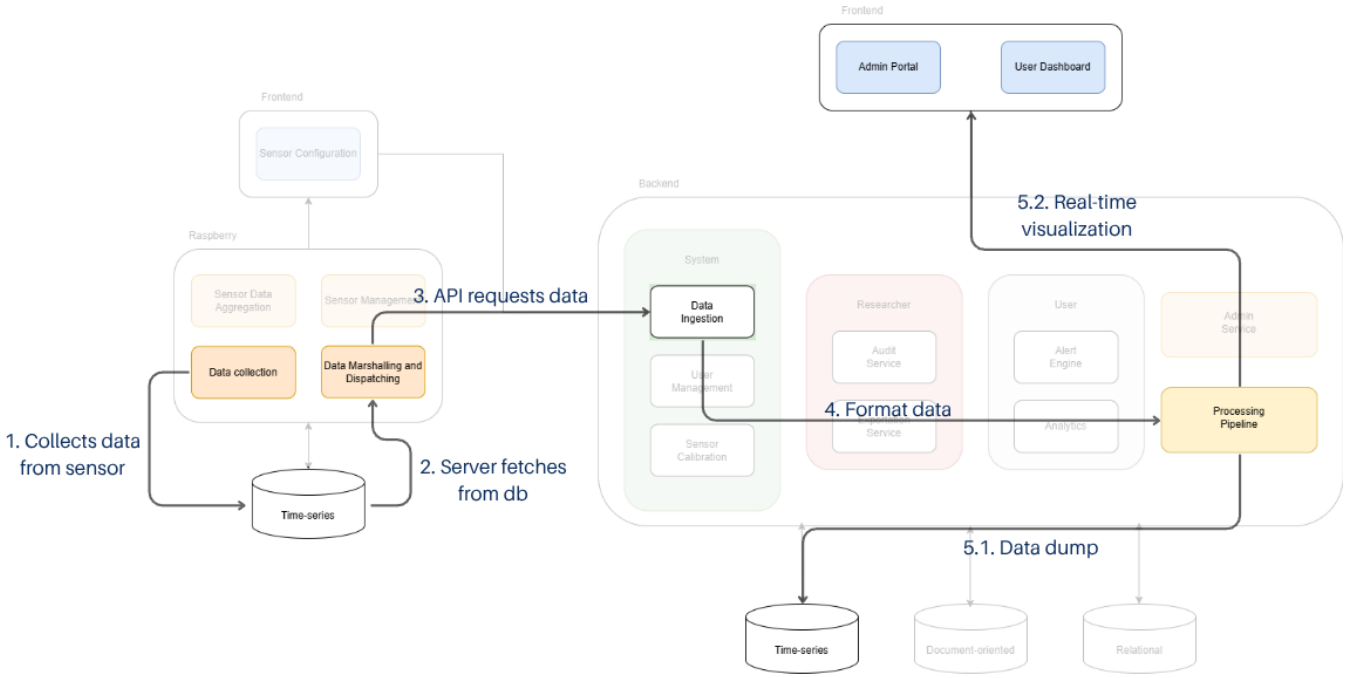


Figure 4.3: Data processing pipeline.

This pipeline structure emphasizes modularity and scalability. The division allows the Raspberry Pi to focus on reliable data capture, the backend to handle storage and complex processing, and the frontend to specialize in user interaction and high-fidelity visualization. This design facilitates independent development and future enhancements, allowing algorithms for posture classification to be integrated into the backend without any modification required to the data acquisition or visualization components.

Data Acquisition and Edge-to-Backend Ingestion

At the edge of the system, the Raspberry Pi handles primary data acquisition, continuously reading data from the IMU network. This raw orientation data is not processed locally for complex analysis. Instead, it is immediately timestamped, formatted into packets and inserted into a local database. Simultaneously, an HTTP server runs on the Pi, exposing this local database through a minimal RESTful API; this design transforms the Raspberry Pi into a dedicated data gateway.

The interaction between the Raspberry Pi (Edge) and the central Backend server is established over a local Wireless Local Area Network (WLAN), currently configured via a dedicated personal hotspot to ensure network isolation and stability.

Server Configuration

The Raspberry Pi hosts a lightweight Flask web server that exposes the sensor state via a RESTful API. The server listens on `0.0.0.0:8080`, making it accessible to any device within the local subnet. The primary endpoint, `/api/sensor`, returns a JSON object containing the timestamped motion vectors (quaternions, linear acceleration) and calibration status.

Data Ingestion Strategy

To retrieve this data, the backend implements a "pull-based" ingestion service. Unlike a pool model, where the sensor dictates the flow, the backend actively polls the Raspberry Pi at a configurable interval (currently set to 1 second). This design choice allows the backend to throttle data consumption based on system load.

Security and Token Verification

To secure the communication channel between the infrastructure nodes, the Flask server running on the Raspberry Pi enforces an API key check on all sensor and recording endpoints via a custom Python decorator (`@require_api_key`). The Pi server reads the `X-MotionSuit-Token` header from incoming HTTP requests and compares it against the secure environment variables, rejecting unauthorized queries instantly.

Central Processing, Storage and Backend-to-Frontend Interface

The backend periodically polls the Raspberry Pi's server API via dedicated endpoints to fetch the latest data from the sensors. After receiving this, the backend performs two primary processes. First, it stores the raw sensor data in the Time-Series database, associating each entry with the relevant user and session for future historical review and aggregate analysis. Second, it prepares the data for real-time visualization by relaying it to the frontend via a WebSocket connection.

To satisfy the requirement for fluid, low-latency visualization on the client side, the communication strategy shifts from HTTP polling to the WebSocket protocol.

Real-Time Streaming

The backend exposes a WebSocket endpoint at `/ws/live`. Upon client connection, the server promotes the HTTP handshake to a persistent TCP connection. As the backend service

injects new frames from the Pi, they are immediately broadcasted to all connected WebSocket clients.

The posture analysis module is implemented at this stage of the pipeline. On every frame received from the Raspberry Pi, the backend applies the selected ergonomic scoring method to the joint angles before relaying the result to the frontend via WebSocket. The resulting score and risk classification are appended directly to the data message, making them immediately available to the dashboard for display and alert evaluation.

Client Integration

The frontend application maintains this connection via a dedicated provider component. It listens for incoming `sensor_data` events and dynamically updates the internal state, triggering an immediate re-render of the 3D exoskeleton visualization. This architecture ensures that the user feedback loop remains synchronized with the physical movements of the wearer with minimal overhead.

Offline Buffered Synchronization (Store-and-Forward)

To support operational scenarios where real-time visualization is not required or network connectivity is intermittent (such as working shifts on a production line), the system implements a "Store-and-Forward" architecture. This allows the wearable unit to function autonomously, buffering data locally before synchronizing with the central backend.

When a recording session is initiated in offline mode, the Raspberry Pi bypasses the WebSocket broadcast and writes sensor frames directly to its local time-series instance. This ensures data persistence even if the central server is unreachable. Once the device reconnects to the network, the Backend initiates a bulk data transfer via the `DataSyncService` class. This process follows a robust three-step protocol:

1. **Session Discovery**: The backend queries the endpoint `/api/dump/sessions` to retrieve a manifest of all completed sessions stored on the edge device.
2. **Paginated Transfer**: To manage network load and memory usage, data is not transferred in a single payload. Instead, the `sync_session` method implements a cursor-based pagination strategy, requesting data in batches (configured to 500 frames per request). This ensures the reliable transfer of large datasets containing hundreds of thousands of frames without overwhelming the application layer.
3. **Schema Alignment & Storage**: Received frames are mapped from the flat JSON structure of the edge device to the rigorous SQLAlchemy models required by the backend's TimescaleDB. Upon successful verification and storage, the backend issues a delete command to the edge device to reclaim storage space, ensuring long-term analysis capabilities without burdening the Raspberry Pi with historical data storage.

4.5 Posture Analysis

The MotionSuit platform supports two standardised ergonomic assessment methods, selectable by the user when starting a monitoring session: REBA, suited for dynamic activities

and standing postures, and ROSA, suited for seated office environments. On every data frame received from the Raspberry Pi, the backend computes the score for the active method and appends the result directly to the WebSocket message sent to the frontend, ensuring the dashboard always reflects the current ergonomic state.

REBA – Rapid Entire Body Assessment

REBA evaluates whole-body postural risk by dividing the body into two groups that are scored independently before being combined. Group A covers the trunk, neck, and legs. Each segment receives a sub-score derived from its current Euler angle: for instance, a trunk flexion between 0° and 20° scores 1, while a flexion above 60° scores 4. Modifiers are applied for lateral bending or twisting. The three sub-scores are cross-referenced against Table A to produce Score A. Group B covers the upper arm, lower arm, and wrist. The upper arm score ranges from 1, when the arm is within ±20° of neutral, to 6, when raised above 90°, with modifiers for shoulder elevation or abduction. The lower arm scores 1 if the elbow is between 60° and 100° of flexion, and 2 otherwise. The three sub-scores are cross-referenced against Table B to produce Score B. Score A and Score B are then combined through Table C to yield the final REBA score, which maps to five risk levels ranging from Negligible (1) to Very High (11+). A score of 4 or above triggers a warning alert, and a score of 8 or above triggers a critical alert. The individual sub-scores are also inspected to identify the primary source of risk, so each alert message includes a specific corrective action targeting the most affected body segment.

ROSA – Rapid Office Strain Assessment

ROSA is designed for office-based postures and evaluates ergonomic risk across three independent sections before combining them into a final score. Section A assesses chair ergonomics. Section B evaluates the monitor and telephone setup based on neck flexion and lateral bend. Section C covers peripheral device usage through shoulder abduction and wrist extension angles. The three section scores are progressively combined through dedicated lookup tables to produce the final ROSA score, which produces a score ranging from 1 to 10, mapped to three risk levels: Low (1–4), Medium (5), and High (6–10). It is important to note that the full ROSA worksheet requires several workstation parameters that cannot be measured by IMU sensors alone. Properties such as chair seat height, seat pan depth, and monitor distance are physical characteristics of the environment rather than body angles. In the current implementation, these values are either approximated from available sensor readings, for example, trunk angle is used as a proxy for back support quality and knee angle as a proxy for seat height, or set to neutral defaults. As a result, the ROSA score reflects the user's postural behaviour in a seated position rather than a complete workstation ergonomic audit. A score of 4 or above triggers a warning alert, and a score of 7 or above triggers a critical alert.

Alert Integration and Historical Analysis

Both scoring methods feed directly into the real-time alert engine. After each score is computed, the system checks whether it exceeds the method-specific threshold and whether a 15-second cooldown since the last alert has elapsed, preventing repeated notifications for a sustained harmful posture. An inactivity condition runs in parallel: the system continuously tracks aggregate joint movement between frames. If no significant movement is detected for more than one hour, a movement reminder alert is issued regardless of the current posture score, prompting the user to stand up and stretch. When an alert is triggered, it is persisted to the database, assigned a severity of *warning* or *critical*, and appended to the outgoing WebSocket frame so that the frontend can display it immediately alongside the live sensor data.

Beyond live feedback, the platform exposes the accumulated posture data through a dedicated analysis page available to every Suit User. The page organises historical data across days, weeks, months, and years, each providing a breakdown of the sessions recorded within that period. For each time window, the dashboard presents three key metrics derived from the stored session records: the total number of alerts triggered, the cumulative time spent wearing the suit, and the evolution of the posture score across individual sessions. This allows users to identify trends over time, such as a gradual improvement in average REBA or ROSA score across weeks, or recurring alert spikes on specific days, providing context that a single live session cannot offer.

4.6 Data Storage and Export Ecosystem

The platform utilizes a hybrid polyglot database strategy combined with an independent data management microservice to decouple synchronous user metadata transactions from heavy asynchronous time-series ingestion and analytical file exports.

Each storage engine was selected and configured to align precisely with the underlying nature of the system data structures it manages:

- **Relational Storage**: Holds all highly structured transactional application schemas, including verified user profiles, cryptographic account permissions, gamification progression tables, alert history logs, and quiet hours notification routing rules.
- **Time-Series Storage**: Optimized for high-frequency sensor readings, storing complete sensor arrays per frame including quaternion vectors, linear acceleration, gyroscopic data, calibration flags, and ergonomic evaluation sub-scores.
- **Document-Oriented Storage**: Houses individualized suit calibration documents. Each record contains the full per-sensor quaternion transformation offset matrices computed during the user's calibration session, allowing the application to dynamically upsert correction frames on every recalibration cycle.

Data Export Architecture

The Data Export Service follows a strict layered architecture, separating data validation (Schema), request routing (Router), and business logic (Service) to guarantee reliable file

extractions without server performance depletion.

The entry point is a single POST endpoint accepting an `ExportRequest` schema that normalizes parameter parameters, including target scopes (e.g., "sensors", "sessions", "alerts"), relative time windows (7, 30, 90 days, or all-time), and file types (CSV, JSON, PDF). The routing layer intercepts requests and injects the corresponding database dependency based on data mapping constraints, routing profile queries to PostgreSQL and telemetry data to TimescaleDB.

To secure server memory resources when creating extensive files under heavy loads, the router completely avoids loading the full query response into RAM. Instead, it utilizes a `StreamingResponse` wrapper, programmatically streaming generated binary lines directly to the client network socket as they are retrieved. The core logic executes hierarchical filtering (filtering first by specific `session_id`, then by localized user context, and lastly by time bounds) to transform raw database objects into standard file models:

- **CSV**: Compiled using the Python `csv` library combined with model introspection to automatically map row metrics to tabular headers.
- **JSON**: Generated by serializing active queries into structured hierarchical dictionary lists.
- **PDF**: Created dynamically via the `reportlab` engine, structuring summaries, timestamps, and data blocks into readable documents.

4.7 Dashboard Development

The development of the MotionSuit frontend followed a user-centered design methodology, prioritizing usability and adherence to the functional and non-functional requirements defined in Chapter 3. The implementation leveraged modern web technologies to ensure responsiveness and performance. The process was executed in three distinct phases to ensure alignment between the initial vision and the final technical implementation.

Development Methodology

The construction of the interface followed a "Design-First" approach, divided into the following stages:

1. **Prototyping (Figma):** High-fidelity mockups were created in Figma to validate the visual hierarchy, color palette, and navigation flows for each actor, without writing code.
2. **Static Implementation:** Once the designs were approved, the frontend was developed as a static React application. During this phase, the interface components were built using mock data, allowing the frontend development to proceed in parallel with the backend API construction.
3. **API Integration and Refinement:** After the backend endpoints were established, the frontend was connected to the live API using asynchronous data fetching. This integration phase created a feedback loop, identifying necessary adjustments in the backend API response structures to better serve the UI requirements.

The 3D Skeleton Visualization

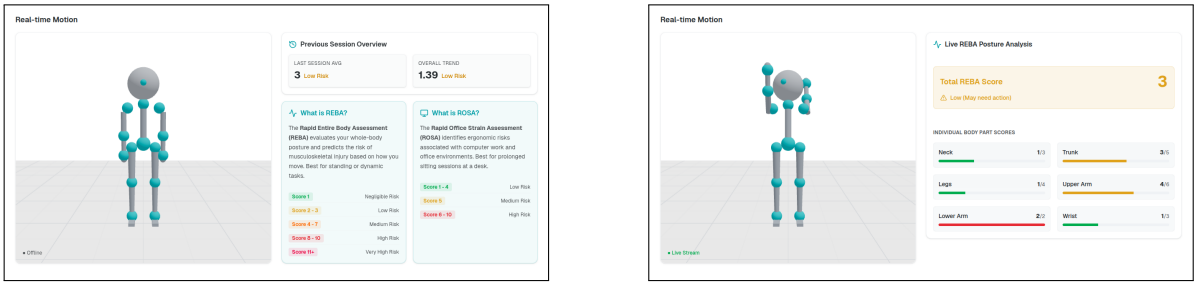
The centerpiece of the user interface is a 3D skeletal model implemented to provide the user with real-time feedback. Its primary purpose is to translate the raw orientation data coming from the sensors into an intuitive visual representation, allowing users to see a digital replica of their own body movements as they occur. This approach is critical for bridging the gap between numerical sensor data and human awareness, making harmful postures immediately recognisable.

The model represents the full human body across 14 joints, covering the neck, lower back, both shoulders, elbows and wrists, both hips, knees and ankles. Each joint is a node in a hierarchical scene graph, so rotations propagate through the skeleton the same way they do in the physical body. Every frame received over the WebSocket carries three-axis orientation data for each joint expressed as pitch, roll, and heading angles, which are applied directly to the corresponding node so the figure continuously mirrors the user's actual posture.

A meaningful update relative to the earlier version of the system is posture-aware positioning. When the active ergonomic method is ROSA, designed for seated office environments, the model smoothly transitions into a seated pose. This keeps the visual feedback consistent with the analysis context without any abrupt change in the model's appearance.

Alongside the 3D viewer, a live analysis panel displays the ergonomic scores for the active method. In REBA mode, the panel shows the total score and its risk level alongside the individual sub-scores for each body part: neck, trunk, legs, upper arm, lower arm, and wrist. In ROSA mode, it shows the total score and the breakdown across the three workstation sections: chair ergonomics, monitor and telephone, and mouse and keyboard. Each sub-score is accompanied by a colour-coded bar that fills relative to its maximum, shifting from green through amber to red, so users can immediately see which body regions or workstation components are contributing most to their current risk level.

Figure 4.4 demonstrates the model in two states: a static representation when no session is active and a dynamic mode responding to live movement data.



(a) Static orientation of the 3D skeletal model. **(b)** Dynamic posture replication.

Figure 4.4: Comparison of the 3D model in static and simulated movement modes.

Suit Calibration Workflow

Before starting the first session, the user is required to perform a calibration step to establish a personal baseline. Each BNO055 sensor has its own internal reference frame

determined by how it was physically mounted on the suit, meaning that even when the user stands in a perfectly neutral position, the sensors will not all read zero rotation. Calibration measures this mounting offset for each sensor so it can be corrected in all subsequent readings.

The process is guided through a modal dialog and consists of a single pose: the user stands upright with their arms relaxed at their sides, known as the N-Pose. Once the user confirms they are ready, a five-second countdown runs, after which the frontend sends a capture request to the backend. The backend fetches a live sensor frame directly from the Raspberry Pi and stores the raw quaternion values from all 14 joints in memory.

When the capture completes, a finalization request is issued. The backend then computes a rotation offset for each joint by comparing the captured quaternion against the expected neutral quaternion for that position. These offsets are calculated using quaternion inversion, producing a correction rotation that, when applied to future readings, effectively zeroes out the sensor's physical mounting bias for that user. The resulting calibration profile is stored in MongoDB and associated with the user's account, so it can be applied automatically in subsequent sessions. The calibration can be repeated at any time from the Settings page.

Access Interface

The Access Interface serves as the centralized entry point for the MotionSuit platform, balancing security with ease of access. The authentication process is consolidated into a unified Login page capable of routing all distinct user roles to their respective environments upon authentication. Complementing this, the integration process is customized through dedicated Registration pages for Suit Users and Researchers, ensuring that the specific data requirements for each profile are captured efficiently during account creation.

Mock-ups

The design of the access interface focused on simplicity and clarity to ensure a smooth integration process for all user roles. The mock-ups established a clean layout incorporating the previously defined color palette and logo to reinforce brand identity. For the Login screen, the design prioritized a minimal form requiring only credentials such as email and password, aiming to reduce cognitive load. The Registration interface was designed to be equally intuitive, including real-time validation feedback for password strength and required fields, ensuring suit users and researchers could create accounts without friction.

The initial Figma mock-ups for the login and multi-role registration flows are presented in Figure 4.5.

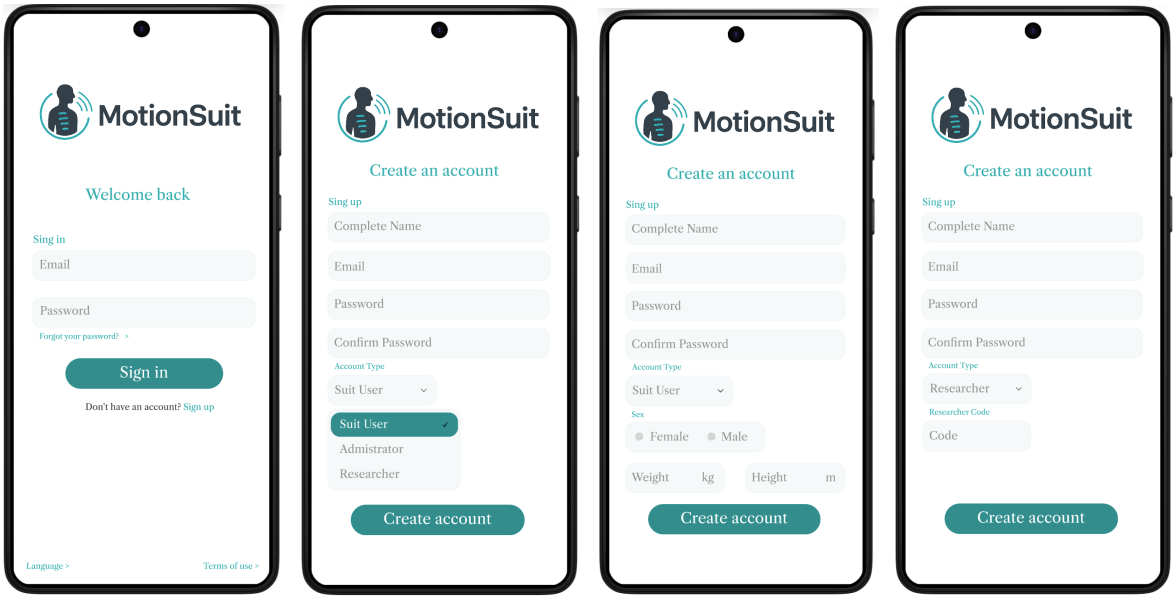


Figure 4.5: Access interface mock-ups.

Implemented Authentication Architecture

The final implementation of the Access Interface establishes a secure, unified authentication gateway that integrates the Keycloak Identity Provider (IdP) with the local system storage.

To maintain a cohesive visual identity, all authentication pages (login, registration, forgot password, and email verification prompt) were customized using `Keycloakify`. This utility compiles a React-based frontend theme into a Keycloak-compatible theme archive. This ensures the login pages inherit the exact same HSL color palette, typography, layout, and responsive styling as the primary Next.js dashboards.

Registration enforces a two-step verification sequence. When a new user wants to register a new account:

1. **Credential Entry:** The user submits their email, username, and password through the themed registration interface.
2. **Email Verification:** Keycloak blocks account activation and automatically dispatches a verification email containing a secure, time-limited link. The account is only enabled once the user clicks this link, protecting the system from spam registrations.

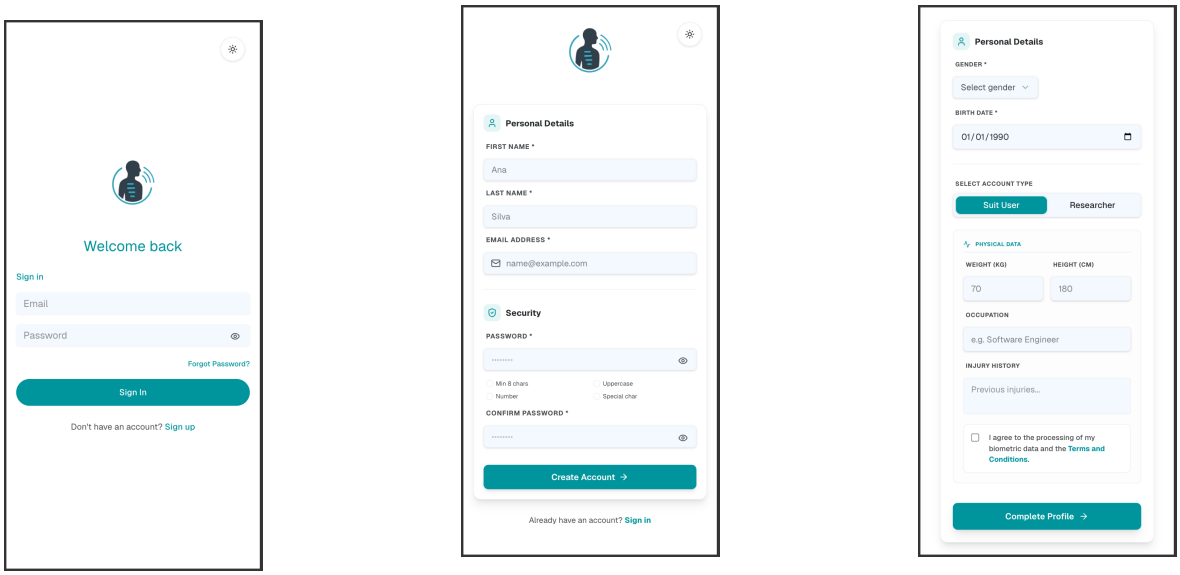
Upon successful email verification and first login, the user is redirected to a mandatory onboarding wizard. Because Keycloak serves strictly as an identity store, the system requires domain-specific metrics. The onboarding page collects user metadata (height, weight, occupation, and Terms & Conditions acceptance) and sends a POST request to the backend completed profile endpoint (`/users/profile/complete`). This updates the local PostgreSQL database, aligning Keycloak's UUID with a local user record.

The "Forgot Password" feature is managed via Keycloak's secure mailer flow. When triggered, Keycloak sends a cryptographically signed link to the user's email. Clicking the

link redirects the user to the custom-themed password reset page, allowing them to define a new credential without the backend ever handling or exposing plaintext passwords.

To safeguard the system from default credential exploits, the administrator account is provisioned with a "Required Action" flag in Keycloak. Upon the administrator's initial login, Keycloak intercepts the session and forces a mandatory password change before granting access to the control panel, ensuring that default deployment credentials are never left active.

The final versions of the Login and Registration screens that were implemented with keycloak, are displayed in Figure 4.6.



(a) Login Interface.

(b) User Registration Interface.

(c) Complete Profile Interface.

Figure 4.6: Custom Keycloak Access interfaces.

Suit User Interface

The Suit User Interface serves as the direct feedback loop for individuals wearing the MotionSuit device. Its primary objective is to translate raw sensor data into immediate, actionable insights regarding postural health. The interface is designed to support two distinct operational states: active monitoring, where real-time visualization facilitates self-correction and historical review, which allows users to manage their data and track progress over time.

Mock-ups

The conceptual design focused on a mobile-first, gamified approach to maximize user engagement. The mock-ups prioritized a high-level "Posture Score" for quick assessment, supported by simplified progress bars for joint angles and daily goal tracking. The design also established the workflows for device pairing via ID, notification management, and a configuration system for exporting data in various formats.

These design concepts, illustrating the intended mobile experience, are shown in Figure 4.7.

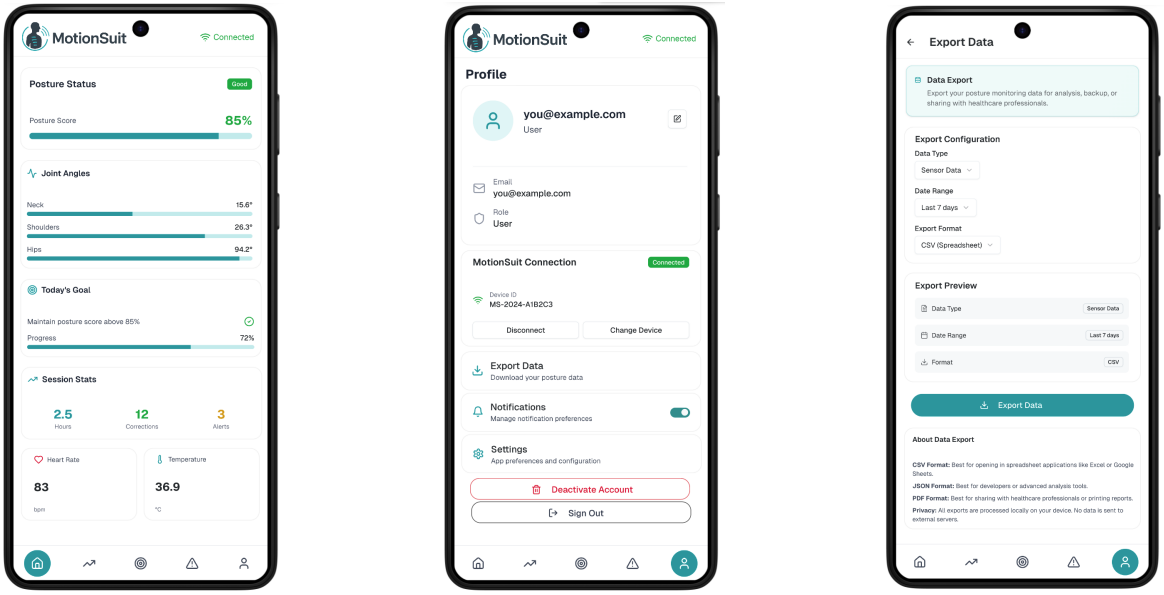


Figure 4.7: Suit User Interface mock-ups.

The final Suit User interface transforms the platform into a complete ergonomic analysis and user engagement solution. The integration of the full MotionSuit hardware configuration, incorporating all fourteen BNO055 sensors, allows the 3D skeletal model to accurately replicate the user's body movements in real time, with live ergonomic analysis performed continuously as new sensor data is received. REBA and ROSA scores are calculated on each incoming frame, providing immediate posture assessment. During simulation mode, users can manually select which ergonomic evaluation method to apply, whereas in real-world operation the system automatically switches to ROSA analysis whenever a seated posture is detected.

User Dashboard

When no session is active, the dashboard displays a static 3D skeleton in a neutral standing pose, along with an overview of the user's most recent session and the overall trend across all past sessions. A brief explanation of the two ergonomic scoring methods is provided, REBA is suited for dynamic activities and standing postures, while ROSA is designed for seated office environments, with each score mapped to a corresponding risk level. When a real-time session is started, the 3D skeleton mirrors the user's actual movements, and the interface shows live ergonomic scores for each body segment (e.g., trunk, neck, ...). Additionally, the dashboard integrates JARVIS, an intelligent conversational assistant that helps users interact with the system using natural language. Users can ask questions about their posture data, request session summaries, or inquire about system features, all of which are answered with context-aware responses in real time.

Session Analysis

Allows users to review their past sessions using time-range filters such as days, weeks, months, or years. For each selected period, the page displays aggregated metrics including

the average REBA/ROSA score, total hours spent wearing the suit, number of posture alerts needed, and the total alerts generated. Users can access a complete list of all sessions that match the selected filter. Selecting any individual session opens detailed line charts that visualize how the REBA or ROSA score changed throughout that session, enabling users to identify exactly when risky postures occurred and to monitor their improvement across multiple sessions. A Posture Trends section highlights the session with the lowest risk score (Best Session) and the session with the highest risk score (Worst Session).

Gamification System

Gamification was added to increase user engagement and encourage healthy postural habits. Users earn experience points (XP) and progress through global levels, base XP is awarded for each completed session, with a bonus for maintaining low-risk posture scores. Cumulative XP drives a level-up system that triggers automated notifications upon reaching new ranks. Daily activity streaks are tracked and rewarded with XP bonuses, while multipliers incentivise movement during historically low-activity periods. Weekly goals, generated from a deterministic seed so that all users see the same challenge at the same time, keep users engaged, in parallel, custom goals allow users to personalise their progression by defining specific metrics, difficulty tiers, and deadlines. Finally, long-term milestones unlock unique one-time badges that provide a permanent visual roadmap of achievements, and all gamification data is compiled into a weekly summary report and an automated report for active users.

Alerts System

Organises all posture-related notifications into three distinct categories. Critical alerts indicate a very high-risk posture requiring immediate correction to prevent injury. Warning alerts correspond to moderate risk, advising the user to adjust their posture soon. Info alerts are purely informational for example, a "time to stand up and stretch" reminder after prolonged inactivity, or a notification about a newly unlocked achievement. Users can view the complete history of all alerts, filter the list by any of the three categories to focus on specific types, and further restrict the view to only unread alerts. Each alert can be marked as read individually, or the user can mark all alerts as read with a single action, keeping the notification list clean and up to date. Users can also delete their entire notification history if they no longer find it necessary to keep it.

Profile & Settings Control

Users can manage their personal details (excluding email) and control their connection to a suit or simulation. By accessing the Settings tab, users can personalize their system experience and configure their suit calibration settings. They can select their preferred notification mode and enable the Quiet Hours mode to mute non-critical alerts. They also have the option to allow or disallow the sharing of their data for research purposes and manage accessibility and usability preferences, such as Colorblind Mode and Text-to-Speech. Users can also review the Terms and Conditions accepted during registration and permanently delete their account if desired. The Data Export tab allows users to export their session data by selecting a date

range (last 7, 30, or 90 days, or all-time) and a preferred export format (CSV, JSON, or PDF). Before exporting, users can see a preview of what will be exported and how. Additionally, the Session History tab provides a complete record of past sessions, organized by date and time, displaying each session's duration and the number of frames captured. Users can search for specific sessions using either the session date or session ID, export individual sessions in CSV, JSON, or PDF format, and permanently delete session records when required.

Figure 4.8 presents the final implementation of the dashboard and Figure 4.9 the analysis, gamification and alerts tools that were added for the suit user.

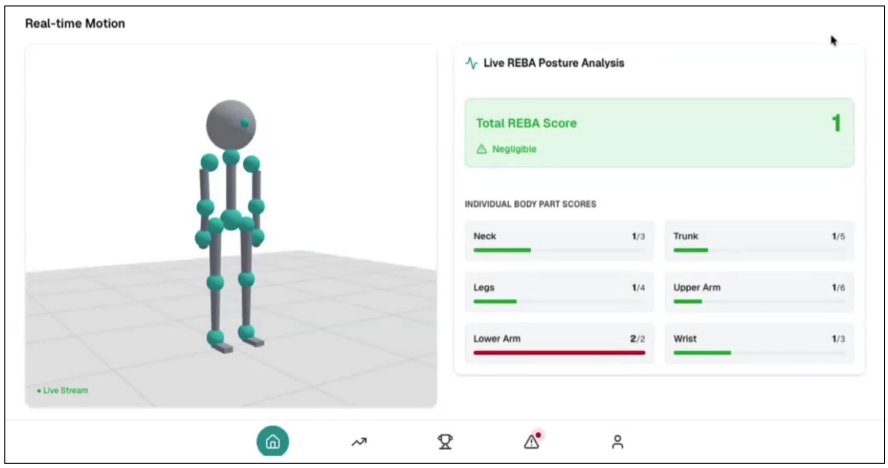
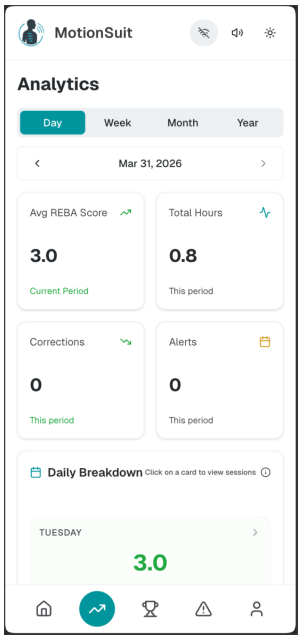
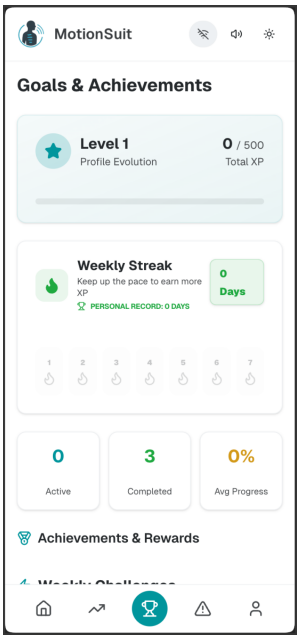


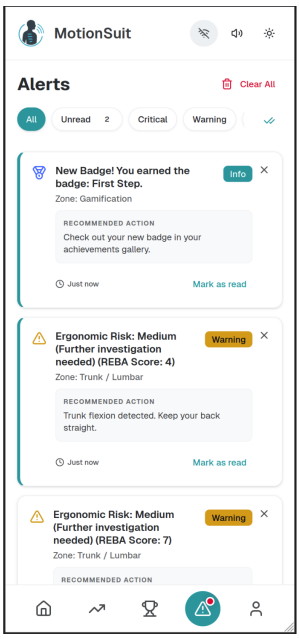
Figure 4.8: Final User Dashboard.



(a) Session Analysis Page.



(b) Gamification Page.



(c) Alerts Page.

Figure 4.9: Suit User secondary interfaces (Analysis, Gamification, and Alerts).

Administrator Interface

The Administrator Interface serves as the centralized control tower for the MotionSuit platform, designed to ensure system integrity, user oversight, and infrastructure stability.

Mock-ups

The administrative interface was architected around five distinct navigation pillars, strategically designed to ensure a clear separation of concerns and a streamlined operational workflow. The Overview module is dedicated to immediate system health checks, providing a high-level summary of active metrics and service status in real-time. The management of users and physical assets is divided between the Users section, which governs the lifecycle of accounts and access control, and Suits, which serves as a centralized hardware inventory for tracking the deployment and pairing of individual Motion Suit units. The visual design prioritized "management at a glance", using large data cards, clear status indicators and color-coded alerts to help administrators instantly identify critical issues or outdated firmware.

The mock-ups for the administrative command center and its monitoring modules are depicted in Figure 4.10.

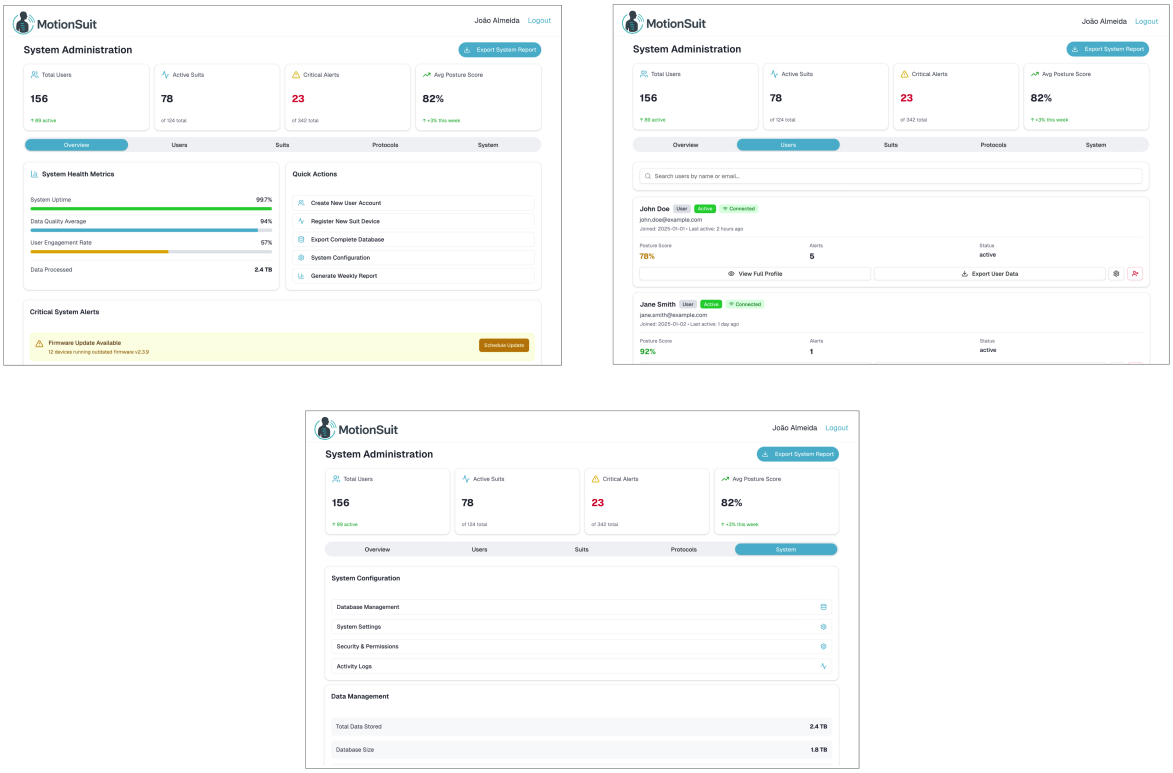


Figure 4.10: Administrator Interface mock-ups.

Product Architecture and Workflows

The final implementation of the Administrator Interface translates the mock-ups into a production-ready Web Dashboard integrated with Keycloak OIDC services and the FastAPI backend.

All administrative user management functions are backed by the Keycloak Admin API, integrated into the FastAPI backend via an asynchronous Keycloak client wrapper. The frontend dashboard retrieves user states dynamically, ensuring that authentication privileges, credentials, and roles are securely synchronized.

When an administrator deletes a user account (or when a user requests self-deletion), the system executes a coordinated two-phase deletion protocol to maintain security while preserving database references:

1. **Identity Revocation (Hard Delete):** The backend issues an asynchronous HTTP DELETE request to the Keycloak User Management endpoint `/auth/admin/realms/{realm}/users/{user_id}`. Keycloak immediately deletes the user's security principal, revokes all active access/refresh tokens, and terminates their active sessions, preventing any further system access.
2. **Data Consistency (Soft Delete):** Because timeseries motion telemetry stored in TimescaleDB is linked to the user's unique identifier (UUID) via foreign key relationships, a cascading database delete would wipe out historical scientific data. To prevent this, the backend executes a soft delete locally in the PostgreSQL database. The user record is updated with a `deleted_at` timestamp. Any future query for active users filters out these "tombstoned" records, keeping personal details hidden while maintaining timeseries integrity.

To prevent unauthorized access to sensitive participant telemetry, the registration of clinical researchers follows a strict approval gateway managed by the administrator:

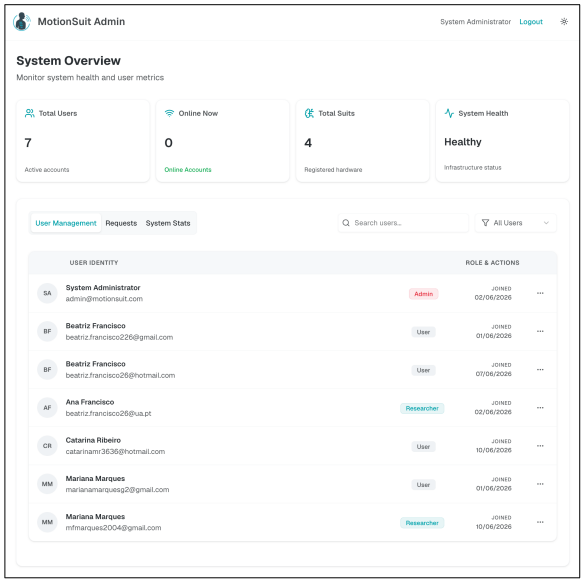
1. **Pending State:** During registration, a researcher provides their academic institution and area of interest. The backend stores this metadata in a local PostgreSQL table and assigns the user the `pending_researcher` role in Keycloak.
2. **Review Directory:** The Admin Dashboard calls the backend endpoint `/admin/pending-researchers`. The backend queries Keycloak for all members of the `pending_researcher` role, joins these accounts with the local institution details, and displays them in a review queue.
3. **Approval Action:** If the administrator approves the request, the backend calls Keycloak to remove the `pending_researcher` role and assign the active `researcher` role. Locally, the system records the approval timestamp (`approved_at`) and the administrator's ID (`approved_by`) to create an audit trail.
4. **Rejection Action:** If rejected, the backend calls Keycloak to strip the `pending_researcher` role and assign the `denied_researcher` role. Locally, the system cleanses the database by deleting the researcher's institution profile record from the database.

Serving as the operational command center, this interface exposes specific management workspaces:

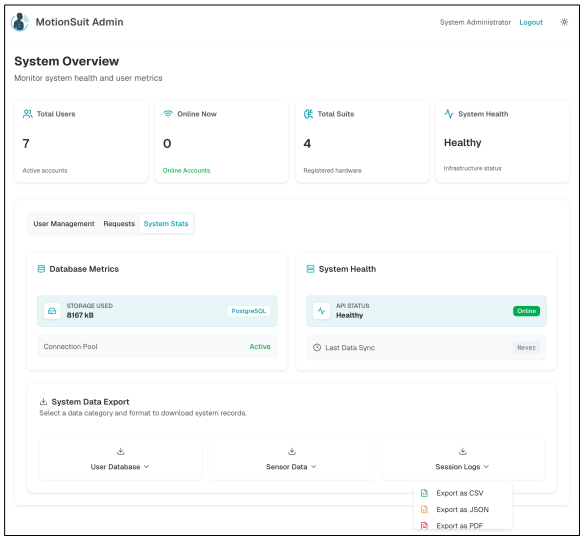
- **System Monitoring:** Displays real-time metrics regarding system health, active user counts, users availability (online/offline), real-time suit usage statistics and PostgreSQL database storage status.

- **User Management:** Enforces the exclusive rights to manage accounts, toggle permissions, and approve/reject researcher request pipelines described above.
- **Suit User Oversight:** Allows administrators to access the full profile details of any specific user, inspect their complete session history, apply time-based filters, and export datasets for individual auditing.
- **System-Wide Export:** Enables the bulk extraction of the entire platform state, generating full database schema dumps, raw telemetry blocks, and event streams for system backups.

The final administrative dashboard and system statistics page are shown in Figure 4.11.



(a) Final Admin Dashboard.



(b) Final System Stats Interface.

Figure 4.11: Administrator implemented interfaces.

Researcher Interface

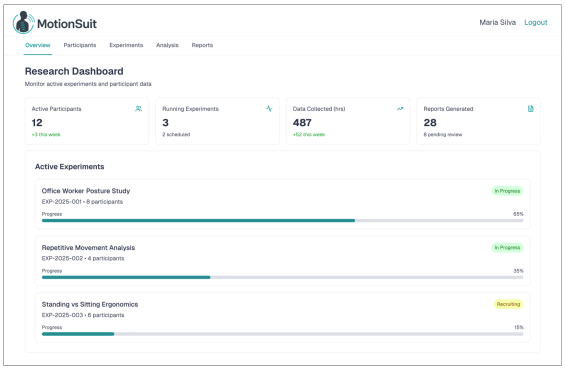
The Researcher Interface is the analytical engine of the MotionSuit platform, designed to allow scientific professionals to manage study groups, extract large amounts of data and interpret biomechanical data.

Mock-ups

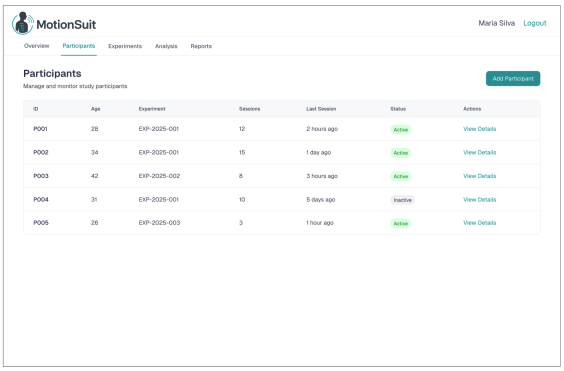
The conceptual design proposed a comprehensive, all-in-one research suite covering the entire study lifecycle. It featured a complex five-module structure intended to allow researchers to conduct data analysis and generate reports directly within the platform, minimizing the need for external software tools.

These ambitious design goals are illustrated in the mock-ups presented in Figure 4.12.

The final implementation of the researcher platform translates the conceptual requirements into a highly unified, five-tab dashboard workspace, as shown in Figure 4.13 . This interface eliminates the need for external processing tools by organizing the operational workflow into five accessible views within the top navigation header:



(a) Researcher Dashboard Mock-up



(b) Participants Management Mock-up

Figure 4.12: Researcher Interface mock-ups.

Overview Tab

Displays quick-info metric cards (active participants, total sessions, total data collection hours, active experiments). Also includes an "Active Experiments" recruitment tracker and a "Recent Activity" timeline.

Participants Tab

Offers a master directory to review and manage subject cohorts. Researchers can filter profiles by age, gender, and occupation. Each row provides two actions: study assignment (move a participant into an active experiment) and a details sidebar. The sidebar shows core metadata (age, height, weight, gender, occupation, consent status) along with medical logs (injury history, medical notes) and a session timeline. From this panel, researchers can export raw data for that specific user. The participants grid executes the core cohort view under a strict anonymity framework. All real names are stripped from the active rows and replaced with encrypted labels (e.g., `Participant OC244891`).

Experiments Tab

Manages research cohorts using cards. Each card displays the trial state, participant recruitment progress, total sessions, and experiment timeline. Clicking "Manage Study" opens a modal to audit enrolled participants, manage cohort memberships, or trigger a bulk CSV export.

Analysis Tab

Provides analytical tools to observe postural trends over time. Users define the scope (global or study-specific) and toggle between two dashboards. The Correlations Dashboard shows chronological graphs of average REBA/ROSA scores per hour and an alert intensity heatmap. The Demographics Dashboard displays REBA/ROSA scores by demographic groups and a posture variability box plot by profession

Reports Tab

Houses the Report Centre with two extraction options and a recent export history ledger. The Cohort Activity Summary (PDF) computes group averages and relative risk analyses for a selected time range. The Scientific Data Extraction (CSV) operates under a prominent "Privacy-by-Design active" flag, allowing investigators to download either detailed telemetry logs (sensor readings) or structured anonymised profile overviews (basic demographics).

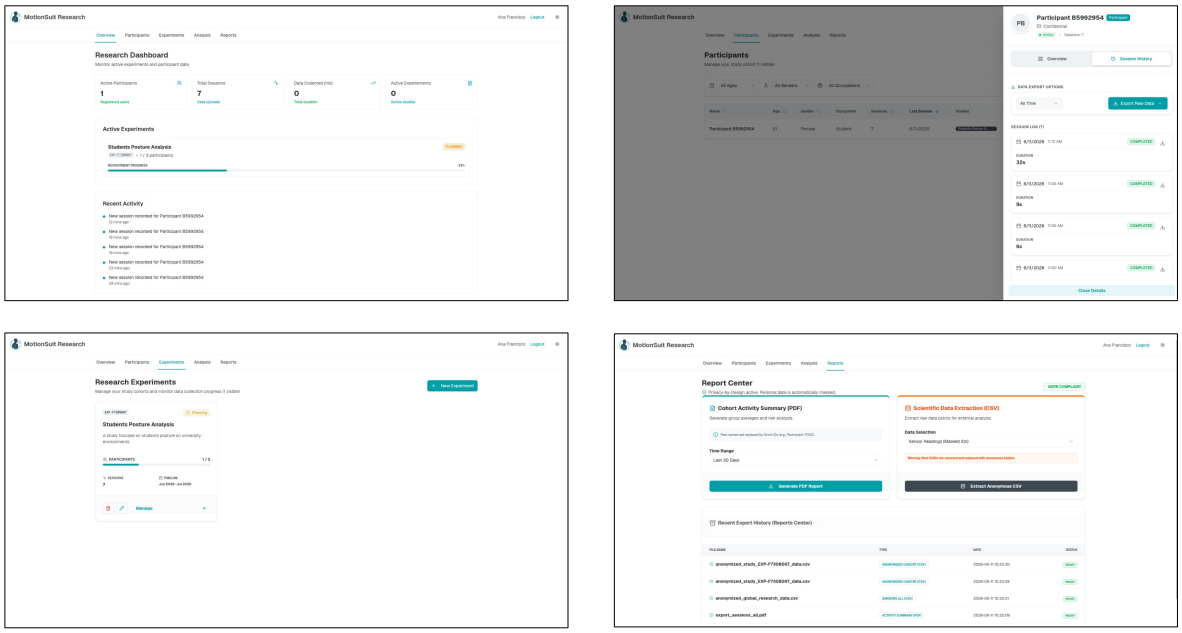


Figure 4.13: Researcher dashboard – Overview, Participants, Experiments, and Reports tabs.

4.8 Usability & Accessibility

To ensure that the MotionSuit platform is usable by a wide range of individuals, special attention was given to usability and accessibility from the early design stages. The frontend implements a set of global providers and dedicated components that address real-world inclusion needs, such as screen reader support, colour blindness adaptation, clear system feedback, graceful error handling, and full responsiveness across devices. All accessibility preferences are stored in the user's backend profile, ensuring a consistent experience across sessions.

Responsiveness and Mobile Adaptation

A mobile-first approach was used to build the MotionSuit frontend, ensuring a consistent experience across desktops, tablets, and smartphones. A custom `useIsMobile` hook was implemented using the `window.matchMedia` API, detecting when the viewport width falls below the 768 px breakpoint. This detection allows the application not only to conditionally apply CSS rules but also to remove non-essential components from the virtual DOM on small screens, improving performance.

Complementing the dynamic hook, extensive use of Tailwind CSS responsive modifiers ensures fluid layout adjustments. For instance, buttons and card containers use the `flex-col sm:flex-row` pattern, stacking elements vertically on mobile and switching to a horizontal alignment once the screen reaches tablet size (640 px or larger). The same strategy is applied to spacing, typography, and grid layouts throughout the dashboard.

System feedback components also adapt to the device context. Toast notifications originating from the `Sonner` library are positioned at the bottom of the screen on mobile devices, making them easily reachable with the user's thumb. On desktop, they appear near the top or the side of the active window, reducing visual interference with the main content. This intelligent positioning, combined with smooth slide animations, provides an intuitive and unobtrusive feedback mechanism regardless of the device in use.

All responsive behaviours were validated across common screen sizes, confirming that the interface remains fully functional and readable without horizontal scrolling or overlapping elements.

Across all pages, a consistent visual and semantic hierarchy is used to guide user attention while avoiding cognitive overload. Each screen follows a simple layered structure consisting of an icon or visual cue, a clear heading (e.g., `<h1>`), a concise descriptive paragraph rendered in muted foreground colours to reduce eye strain, and well-spaced action buttons. This pattern is applied uniformly across dashboards, settings panels, confirmation dialogs, empty states, and error pages, ensuring that users can always understand their current context and interact with the interface intuitively. In addition, semantic HTML elements are properly and consistently labelled throughout the system, preserving full compatibility with screen readers and other assistive technologies.

Error Recovery and System Feedback

To prevent user frustration and avoid the "white screen of death," the system's frontend implements dedicated fallback pages. A custom `not-found.tsx` page presents an empathetic message informing users that a page may have been moved or deleted, along with a clear button to return to the Dashboard. In addition, a global `error.tsx` file functions as an error boundary, catching unexpected client-side exceptions and displaying a friendly error interface instead of allowing the entire application to crash. This error page also includes a `reset()` function that enables users to attempt recovery without manually refreshing the browser, as well as a secondary button that safely redirects them back to the home page. Together, these mechanisms ensure that users are always provided with a clear and reliable path forward, even when unexpected issues occur.

Furthermore, providing immediate and clear feedback for user actions is a core usability principle, implemented through two complementary mechanisms. For lightweight, non-intrusive notifications, the platform uses a toast system built with `Radix UI` and `Sonner`. When users enable an accessibility feature, export data to files, or trigger an error, a small floating notification appears at the top or bottom of the screen depending on the device. Success toasts are styled with a neutral green accent, while error toasts use a destructive red variant

and include a manual close button so they can be dismissed at any time. Although they automatically disappear after a few seconds, they remain accessible to screen readers to ensure full accessibility support.

For higher-stakes actions such as deleting a session, permanently removing an account, or clearing all historical data, the system relies on an alert dialog. This modal component blocks interaction with the rest of the interface using a semi-transparent overlay, ensuring the user remains focused on the decision. Keyboard focus is trapped within the dialog to prevent accidental navigation away. Users must explicitly confirm or cancel the action, reducing the risk of irreversible mistakes. Toasts and alert dialogs provide a consistent feedback system that clearly distinguishes between low-impact warnings and high-impact decisions.

Colour Blind Mode

A global colour blind mode is built into the frontend to accommodate users with colour vision deficiencies, applying an adapted colour palette without forcing unnecessary React re-renders. Upon login, the `useProfile` hook retrieves the user's accessibility preferences from the backend endpoint `/api/v1/users/me`, including the `is_colorblind` flag. The `ColorBlindProvider` listens to this value and, when enabled, directly accesses the root HTML element (`document.documentElement`) to add the CSS class `colorblind-mode`. This class triggers global CSS variables that replace standard colours with a palette optimised for protanopia, deuteranopia, and tritanopia, instantly modifying the entire interface.

Toggling the feature from the settings page does not trigger a full component tree re-render. Instead, a custom event (`"colorblind-toggle"`) is dispatched when the user clicks the switch. The `ColorBlindProvider` listens for this event via `window.addEventListener` and adds or removes the CSS class in real time, while asynchronously updating the preference in the backend through the API. This design guarantees an instantaneous visual change with minimal performance overhead, preserving a smooth and responsive user experience.

Text-to-Speech Support

A comprehensive text-to-speech (TTS) system is integrated into the frontend to assist users with visual impairments or reading difficulties. The implementation is divided into three logical components: mode management, voice engine, and text preprocessing.

Mode Management

The `AccessibilityProvider` supports four distinct TTS modes, each designed for a different usage scenario. A dropdown menu located in the fixed navigation bar allows users to switch between TTS modes from any page, ensuring that accessibility controls remain consistently available throughout the application.

- **Current page**: In this mode, the system reads the currently visible page content aloud exactly once. If the user navigates to a different page, the new page is not read automatically. This mode is useful for users who only need to hear the current screen but do not want continuous reading across multiple pages.

- **Session (All pages)**: This mode reads the current page content aloud and continues to read new pages automatically as the user navigates through the application. However, the preference is not saved to the backend database. This mode is ideal for temporary assistance, for example, when a user has tired eyes or is in a situation where listening is more convenient than reading, but they do not want the setting to persist across future logins.
- **Always Read**: This mode behaves identically to `Session` in terms of real-time reading (it follows navigation), but it persists the user's preference to the backend. When the user enables this mode, the frontend sends a PATCH request to the API setting `tts_enabled: true`. The `useProfile` hook retrieves this value on subsequent logins and automatically activates TTS, ensuring a consistent accessible experience across all sessions without requiring the user to re-enable the feature each time.
- **Off**: Text-to-speech is completely disabled. No content is read aloud, and no speech synthesis resources are loaded. This is the default mode for users who do not require auditory assistance. It also works to deactive the current activated modes including the `Allways Read` setting `tts_enabled: false`.

Voice Engine

The custom `useTextToSpeech` hook bridges the React application with the browser's native Web Speech API to provide accessibility features. To ensure a fluid and reliable user experience, the hook pre-emptively clears any stalled audio queues before initiating playback. Furthermore, the speech rate and pitch are adjusted below the default thresholds to accommodate users with cognitive or hearing difficulties. To produce a more natural and less robotic output, the implementation dynamically scans available system synthesisers, prioritising premium voices such as the `"Google"` engine with an `"en-US"` locale.

Text Preprocessing

The implementation extracts clean, readable text from the DOM without forcing the screen reader to process raw HTML. It isolates the content inside the `<main>` tag, ignoring sidebars and headers, and creates a safe clone of the node to avoid modifying the live interface. Images, scripts, styles, navigation elements, and static `.sr-only` containers are removed. To make the speech flow more natural, the function detects block-level tags (e.g., `<h1>`, `<p>`, `<div>`) and injects a full stop after each one, forcing the TTS engine to insert a breathing pause. Buttons receive the prefix `"Button: "` so that visually impaired users recognise them as interactive elements. Finally, regular expressions eliminate double spaces and format punctuation, ensuring the TTS engine does not stumble over malformed sentence boundaries.

4.9 CI/CD Pipeline and Deployment

The MotionSuit system adopts a fully automated Continuous Integration and Continuous Deployment (CI/CD) strategy implemented through GitHub Actions. Each of the three main subsystems, backend, frontend, and sensor server, has its own independent pipeline, ensuring that changes to one component do not block or interfere with the others.

Continuous Integration

The CI pipeline is triggered on every push or pull request to the `main` and `dev` branches. Before any code can be merged, it must pass a quality gate covering three areas: a dependency security audit, automated testing, and static code analysis.

The security audit runs `pip-audit` on the backend and `pnpm audit` on the frontend to detect known vulnerabilities in third-party packages. Static code analysis is handled by SonarQube, which receives coverage reports from the backend and build output from the frontend to flag quality and security issues.

The backend test suite is written in Python using **Pytest** and is organized into two levels. To keep the tests self-contained, the production databases are replaced with lightweight in-memory equivalents and Keycloak authentication is bypassed by injecting a pre-built test user, so the suite runs in the CI environment without any infrastructure setup.

Unit tests focus on individual components in isolation, patching collaborators so that only the logic under test is actually executed. They span the authentication and permission checks in the core layer, the HTTP status codes and response shapes returned by the API endpoints, and the business logic in the services layer, which includes sensor data transformation, ergonomic score computation, gamification rules, and data export formatting.

Integration tests send real HTTP requests through the full application stack. Each test sets up the specific database state it needs, performs one or more requests, and checks both the response and the resulting database state. Together they cover user and administrator account workflows, alert rule management and data isolation between accounts, sensor session handling, the full gamification flow, posture score computation, researcher-specific features, and privacy behaviour to ensure that user identifiers are never exposed in researcher-facing responses. Tests that require a physical Raspberry Pi are marked separately and excluded from the CI run.

The frontend test suite uses **Vitest** and **React Testing Library** and runs in a simulated browser environment. Tests interact with components by simulating user actions such as clicks and form submissions, with external dependencies like authentication and sensor data providers replaced by controlled mocks. The covered areas include the onboarding and authentication flow, the dashboard posture widgets, the alerts page with filtering and read/unread management, goal and gamification management, profile and settings, the admin panel, and the researcher platform. The frontend CI pipeline validates the codebase through a full production build, which also performs type checking across the entire project.

Continuous Deployment

The CD pipeline is triggered exclusively on pushes to `main`, ensuring only validated code reaches the production environment. It runs in two sequential jobs: first, a Docker image is built for the subsystem and pushed to the GitHub Container Registry under the organisation's namespace; then, an SSH connection is opened to the production server, which pulls the new image and restarts the relevant service.

The production environment is hosted on a virtual machine provisioned on Microsoft Azure. This decision was driven by a practical constraint: the Raspberry Pi must establish an outbound connection to the backend server to stream sensor data, which requires a publicly accessible endpoint with a stable IP address. Hosting on the university's infrastructure would have required registering the Raspberry Pi as an internal university machine, which introduced unnecessary complexity compared to provisioning a straightforward Azure VM.

A further networking challenge emerged from the fact that the Raspberry Pi operates on a local network, while the backend is hosted on a remote Azure VM. Because both nodes must reside within the same logical subnet for the WebSocket and REST communication to function, direct connection was not feasible without exposing public ports. To resolve this, Tailscale was configured as a secure mesh VPN overlay, bridging the two nodes under a private network segment without requiring any open firewall rules on the Azure VM.

4.10 Security Measures

Security is a fundamental design requirement of the MotionSuit platform, ensuring the integrity of telemetry data, the confidentiality of user profiles, and the protection of deployment environments. A comprehensive set of security measures has been integrated across all layers of the system:

Authentication and Access Control (Keycloak)

To enforce robust security at the application layer, the platform integrates Keycloak, an open-source identity and access management solution. Keycloak manages all user accounts, credential storage, and authorization rules.

- **Token-Based Authentication**: The client dashboard authenticates via OpenID Connect (OIDC) to obtain a JSON Web Token (JWT). Every request to the FastAPI backend includes this token in the `Authorization` header.
- **Cryptographic Verification**: The FastAPI backend verifies the authenticity of incoming tokens using Keycloak's public keys, fetched dynamically from the JSON Web Key Set (JWKS) endpoint. This verification checks the token signature, expiration date, and issuer.
- **Role-Based Access Control (RBAC)**: Our system operates with the following roles: `user`, `researcher`, `pending_researcher`, `denied_researcher`, and `administrator` which are managed within Keycloak. The backend inspects the token claims and restricts endpoints accordingly. For example, user data modification requires the `user` role, while running cohort analyses requires the `researcher` role.
- **WebSocket Authorization**: For the real-time telemetry stream (`/ws/live`), the WebSocket connection handshake requires the JWT token as a query parameter. The backend validates the token and compares the token's subject ID (`sub`) with the owner of the active recording session. This prevents unauthorized clients from sniffing live motion data.

- **Secure Authentication Policies**: Keycloak enforces strict identity policies, including two-step email verification, self-service password recovery, and mandatory administrative credential rotation upon first login.

Transport Layer Security (TLS/HTTPS)

All network traffic between system components is encrypted using Transport Layer Security (TLS) to prevent eavesdropping and man-in-the-middle (MITM) attacks.

- **API Endpoints**: The backend is deployed behind an Nginx reverse proxy configured to enforce HTTPS, using industry-standard cipher suites and TLS 1.3.
- **Raspberry Pi Communication**: When the backend pulls data from the wearable suit, the HTTP requests are sent over HTTPS to the Flask server running on the Pi. The backend's HTTP client (`httpx.AsyncClient`) is configured with the CA certificate of the Pi, verifying the Pi's server identity before initiating data transfer.

Edge Access Control (API Keys)

To restrict access to the physical wearable suit even further and prevent unauthorized querying of the sensors:

- **Custom Token Header**: The Flask server running on the Raspberry Pi enforces an API key check on all sensor and recording endpoints via a custom Python decorator (`@require_api_key`).
- **Verification**: The Pi server reads the `X-MotionSuit-Token` header from incoming HTTP requests and compares it against the `PI_API_KEY` environment variable. If the token is missing or incorrect, it immediately rejects the request with a `401 Unauthorized` response.

Secret Scanning and Credential Protection (Gitleaks)

To prevent the accidental exposure of private keys, database passwords, or API tokens in the project's source repositories:

- **Pre-Commit Hooks**: The repositories integrate `pre-commit` hooks configured to run `gitleaks`. Gitleaks scans all staged files for high-entropy strings, passwords, and private certificates before any commit is finalized.
- **Local Enforcements**: If a developer accidentally leaves a credential or API key in a code file, the local Git commit is blocked, requiring the developer to clean the secret from the history before attempting to push.

Dependency Audits and Vulnerability Scanning

To safeguard the system from supply-chain attacks and known software vulnerabilities, the automated GitHub Actions CI/CD pipelines perform security audits on every code push:

- **Backend Audits**: The backend CI pipeline runs `pip-audit` against the `requirements.txt` file, querying the PyPI vulnerability database to detect known Common Vulnerabilities and Exposures (CVEs) in third-party libraries.

- **Frontend Audits**: The frontend CI pipeline runs `pnpm audit -prod` to scan external Node packages for production dependencies.
- **Static Analysis**: Both pipelines upload build metadata and code metrics to SonarQube to perform static application security testing (SAST), highlighting potential security code smells, SQL injection vectors, or cross-site scripting vulnerabilities.

CHAPTER 5

System Analysis

This chapter presents an analysis of the MotionSuit system covering legal compliance, software licensing, regulatory context, identified technical risks and a strategic evaluation of the project's position in the broader market.

5.1 Legal and Ethical Compliance

GDPR and Data Privacy

The MotionSuit system collects 3 categories of sensitive data: personal data (name, gender, date of birth, height, weight), health data (injury history, medical notes) and sensor data. The system was created with protection for that sensitive information and, therefore, there is never a direct connection between a user and their biometric record. To achieve this protection, the identification layer of the MotionSuit uses Keycloak's identity management service for authentication purposes and generating UUIDs as tokens, which ensures that the identification and the processing layers are structurally decoupled from one another, thereby conforming to the EU data minimisation and principles outlined in the General Data Protection Regulation (GDPR).

The application further enforces these strict data masking protocols directly within the user interface, notably inside the **Participants** and **Reports** tabs. Real names are programmatically stripped from the data stream and replaced by unique short identifiers (such as `Participant OC244891`), preventing researchers from establishing a direct link between an individual's civic identity and their sensitive biometric sensor logs.

Regarding data access, researchers may only access the MotionSuit data of users who had previously selected, through their privacy settings, to share their data. Users who have not granted consent to their data being shared will not be viewable in the research interface, no matter their access level as a researcher. Furthermore, the right to delete is enforced at the application level: if a user deletes an account, all records associated will be permanently anonymized, and raw sensor data collected by the user will be automatically purged from the

app after 90 days. All of these retention policies result in a much lower environmental cost of storing inactive data.

In order to protect users' privacy when using the AI assistant, the application runs a local instance of an LLM via Ollama, so the application never sends queries to an external API. This ensures that sensitive health and posture data is never transmitted to third-party services. Finally, before creating an account, all users must read and agree to the Terms and Conditions. These Terms state that MotionSuit is to be used only for personal ergonomic awareness and that does not provide any medical advice.

Software Licenses

The MotionSuit System has different components that are subject to different open source licenses. A green light licensing approach was utilized to ensure the proprietary base remains free of copyleft obligations.

The majority of the stack is covered by permissive licenses. The frontend (React, Next.js and Three.js), backend framework (FastAPI and SQLAlchemy), Adafruit BNO055 drivers and Python requests libraries are all covered by either the MIT license or Apache 2.0 license. Apache has additional protections against patent as it pertains to commercial use. Keycloak and TimescaleDB are covered by either permissive or custom licenses which allow the integration of these technologies without imposing any open source requirements to the other parts of the stack resulting in full integration.

There are two components licensed under the GPL licenses: the database adapter Psycopg2 and the lwm2m library used with the Raspberry Pi, which are isolated from the bulk of the codebase by the use of a containerized modular architecture, providing isolated containers for their operation. This enables the incorporation of GPL software without requiring the whole system to have to be released as an open source project.

Sector Regulation

The system falls under three governing systems. The first, International Organization for Standardization's ISO 9241, requires that feedback from systems be expressed in ways that translate complicated information into an easily recognizable and understandable format. The three-dimensional skeletal visualization meet this requirement by translating actual sensor angles into a "real-time" representation of how the user is positioned.

The second governing system is the European Union Medical Device Regulation (EU MDR) 2017/745, which places strict requirements on all wearable technologies that track or monitor bodily functions. In order to comply with these restrictions, MotionSuit has established a number of data validation procedures to ensure that only correct sensor reading(s) will trigger a user's posture assessment. However, the product is intentionally positioned as a wellness and ergonomic tool rather than a medical device to avoid the full burden of MDR certification during this development phase.

The last governing system is the Engineering Requirement Specification (ERS), which specifies that the feedback generated from MotionSuit must be validated. In order to provide this feedback, the system processes actual sensor data and validates it in back-end processing

before sending posture alerts to users, ensuring that the user will receive reliable feedback based upon verified data instead of erroneous raw sensor readings.

5.2 Technical Risk Assessment

Three areas of accumulated technical debt were identified during this development phase, beginning with a lack of automated testing, as feature implementation was prioritized over test coverage. Unit and integration tests were subsequently implemented to verify that all features function as intended. Additionally, the codebase was reviewed for dead code, with outdated utility functions identified and removed to reduce maintenance overhead.

Fragile Components

Four components were identified as presenting elevated fragility risk, starting with the Keycloak–PostgreSQL synchronization. The onboarding flow relies on an exact mapping between Keycloak tokens and the users table schema, meaning a mismatch between these two would completely block new user registrations. This is mitigated by a service layer that validates data before insertion and allows for schema flexibility. Concurrently, the use of NextAuth.js (v5 Beta) poses a challenge, as the frontend authentication library is currently a beta release whose internal APIs change frequently, introducing the risk of critical login failures following automatic package updates. The mitigation here is strict version pinning in `package.json` combined with integration tests covering the authentication flow. Furthermore, the REBA/ROSA calculation logic introduces fragility because the ergonomic risk scoring algorithms expect a rigid JSON format from the sensor layer. If a sensor transmits a null value, the result could be an incorrect risk score or an API crash, making strict schema validation on sensor payloads required to prevent this. Finally, remote build dependencies create a vulnerability since the frontend build process currently requires an active internet connection to fetch external assets, causing build failures in isolated or offline environments. This is mitigated by migrating all build-time assets to local storage within the project's `public/` directory.

External Dependency Risks

Three external dependencies were assessed for long-term risk, starting with Ollama as the local LLM engine, which carries a risk of discontinuation regarding its integration libraries. This is mitigated by a modular architecture that allows the local AI engine to be replaced without rewriting the service. Concurrently, Keycloak introduces long-term risk due to the complexity of version upgrades and a strong dependence on OpenID Connect support, which is mitigated by using standard OAuth2 protocols throughout, thereby allowing for a smooth migration to an alternative SSO provider if needed. Finally, the use of Google SMTP exposes user email addresses to a third party by relying on Google's email infrastructure, creating a privacy dependency where the long-term mitigation is to operate a self-hosted email server.

SWOT

The MotionSuit system was assessed across four strategic dimensions: strengths in data architecture and edge-ready design, weaknesses related to hardware cost and component availability, opportunities arising from ESG demand and AI integration, and threats from competing technologies and regulatory risk.

Regarding strengths, the system produces structured telemetry from a high-fidelity data architecture, making it suitable for downstream analysis such as data mining. It has been designed specifically for the Raspberry Pi to process and store data at the edge without constant cloud connectivity. This is further reinforced by a robust security infrastructure, comprising Keycloak-managed identity, role-based access control and end-to-end data anonymization, enabling deployment in sensitive environments.

In terms of weaknesses, the primary concern is the cost of scaling hardware to a full-body network of 15 sensors. Additionally, reliance on specific sensors and microcontrollers exposes the system to supply chain disruptions, which could affect component availability during production and scaling.

On the opportunities side, growing corporate interest in ESG compliance and workplace injury reduction provides a natural entry point into the industrial market segment. The existing clean data architecture also positions the system well for future integration with predictive AI and machine learning models, and the rapid growth of telehealth platforms represents a viable distribution channel for remote ergonomic monitoring.

Finally, the key threats include markerless computer vision systems, which are an emerging alternative to wearable tracking. However, they perform poorly in industrial environments where equipment obstructs line of sight. Potential barriers to enterprise adoption also exist due to employee privacy concerns, and any reclassification of the product as a medical device would impose significant regulatory costs under FDA or MDR frameworks.

TOWS Strategic Options

Several strategic combinations emerged from the SWOT analysis. The edge-ready design, combined with growing corporate ESG demand, supports a B2B entry strategy. The system is particularly suited for production environments where Wi-Fi infrastructure is restricted and local data processing is a hard requirement. In these contexts, edge capability is a competitive necessity rather than a differentiator. Additionally, the high-fidelity data architecture combined with the growth of predictive AI creates an opportunity to develop a machine learning module that operates directly on the collected telemetry, enabling real-time posture correction feedback without external data pipelines.

This technical readiness is supported by the fact that the secure infrastructure directly addresses worker privacy concerns: all data is encrypted, anonymized, and user-controlled. Users can operate the system without consenting to data sharing, which eliminates the risk of the product being perceived as a corporate surveillance tool. Finally, to manage both supply

chain risk and the threat of medical device regulation simultaneously, the product should initially be positioned and marketed as a workplace wellness and sports performance tool. This framing successfully avoids the medical device label and the associated FDA or MDR certification requirements, allowing the project to scale commercially before engaging with the regulatory approval process.

PESTEL Analysis

Many macro-environmental factors influence the system, starting with the political and legal landscape where GDPR and EU data sovereignty policies reinforce the architectural decision to process biometric data locally. The local Ollama deployment and Keycloak-managed identity ensure that sensitive data remains within the user's environment. However, the EU AI Act introduces a future risk: if the JARVIS assistant is classified as a high-risk AI system due to its use of biometric and health data, it would require expensive conformity assessments and human-in-the-loop supervision. To avoid this classification, the system must be positioned strictly as an advisory tool. Additionally, emerging labor directives restricting AI-based worker surveillance may require removing inactivity tracking features to ensure the suit is used strictly for ergonomic purposes.

From an economic and social perspective, the Docker-based, cloud-agnostic deployment architecture allows the system to migrate between cloud providers if cost conditions change, reducing exposure to currency risk and infrastructure price volatility. Furthermore, the local AI strategy eliminates pay-per-token API costs entirely, fixing the AI cost to server energy and maintenance rather than usage volume. This cost efficiency supports the deployment across diverse social demographics, addressing the significant gap in digital literacy that exists between younger, technology-native users and older industrial workers. Accessibility features, including text-to-speech, a colorblind mode and haptic alerts, reduce dependency on screen interaction, making the system usable across this range. Simultaneously, the gamification system, with XP, streaks, and badges, targets the quantified-self engagement patterns of younger workers, while the JARVIS assistant provides a simpler, conversational interface for older users.

Finally, the technological and environmental dimensions rely on a core stack of FastAPI, PostgreSQL and TimescaleDB, providing a stable foundation for handling high-frequency sensor time-series data. Experimental components such as the local LLM are isolated in separate Docker containers, allowing AI model swaps without disrupting the core system, while the decoupled architecture and standardized JSON payloads ensure that a future hardware revision, such as a 5G or NB-IoT sensor network, can be integrated without modifying the backend API. Environmental efficiency is achieved because TimescaleDB's automated compression and retention policies reduce the storage footprint of long-term data, lowering the energy cost of hosting historical sensor records. This is reinforced at the hardware level, where batch processing of 500 samples per synchronization cycle and inactivity detection triggers successfully reduce both network overhead and device battery consumption.

5.4 Usability Testing

A formal usability study was conducted with ten participants to evaluate the usability of the MotionSuit platform across all user roles. During the study, each participant completed a set of representative tasks for the Suit User, Administrator, and Researcher roles, while an observer collected both quantitative and qualitative metrics. The evaluation followed a structured methodology based on three complementary instruments.

Instruments

Three distinct forms were used to collect data throughout the study:

- **SUS Pre-test Questionnaire:** Collected informed consent and demographic information (gender, age, height, weight, health conditions, prior experience with similar systems).
- **Usability Test Tasks Form:** For each task, participants completed it independently and rated its perceived difficulty on a 5-point scale, where 1 represented "very difficult" and 5 represented "very easy". At the end, participants could provide additional observations about the most challenging aspects and suggest improvements.
- **Observer Log Table:** For each task, the observer recorded task completion, correctness, time required, number of errors, whether the participant appeared lost, whether assistance was requested, and an observed difficulty rating on a 5-point scale, the same used on Usability Test Tasks Form.

After the practical tasks, participants completed the full System Usability Scale (SUS) questionnaire, which asked about their overall experience, perceived ease of use, intuitiveness, implementation quality, feature coherence, and whether they would need training or external help to use the system. A final open-ended section allowed additional comments. A entire session lasted approximately 45 minutes per participant.

Suit User Tasks

- Task 1: Create a new suit user account and set up the biometric profile.
- Task 2: Connect a suit to the system and start a real-time monitoring session.
- Task 3: Go to the Dashboard, observe the data and finish the session.
- Task 4: Access the Session History and export the last session.
- Task 5: Export sessions from the last 7 days in CSV format.
- Task 6: Enable "Research Participation" option.

Administrator Tasks

- Task 1: Create a new researcher account and check the account status.
- Task 2: Review the list of pending researchers and approve the new researcher account request.
- Task 3: Change the password for the researcher account.
- Task 4: Verify the details of the suit user account and export the session data.
- Task 5: Check the system status and export the sessions logs in JSON format.

Researcher Tasks

- Task 1: Login in the researcher account created check the information on the dashboard.
- Task 2: Check the participants list and filter it by age from 18 to 25 years in ascending order.
- Task 3: Search for the suit user account created, check the details and export all data in CSV format.

Feedback and Improvements

After compiling the usability test results, all qualitative feedback from the ten participants was reviewed and, where applicable, directly implemented into the final version of the platform. The following table summarises the most relevant observations and the corrective actions taken.

Table 5.1: Summary of usability feedback and corresponding improvements.

Feedback / Observation	**Implemented Improvement**
Icons in the interface lacked explanatory labels.	All icon-only buttons now include tooltips or visible text labels for clarity.
No confirmation prompt when disconnecting the suit during a live session.	Added an alert dialog asking for confirmation before disconnecting, preventing accidental disconnections.
Connecting the suit from the user dashboard required going to the profile page, all users expected a dropdown menu or an interactive control on the "Disconnected" badge in the navigation bar.	The badge was redesigned as a button that opens a modal, allowing users to connect or disconnect the suit without having to leave their current page.
"Deactivate account" was placed just below "Logout" in the Profile page.	Moved the deactivate account option into the Settings page, to make the user only find it if wanted as suggested.
Password strength dots (shown before typing) on the registration page had a colour too similar to the background.	Changed the inactive dot colour to a lighter grey for better contrast.
On the researcher participants table, the "View" button was confusing, "Details" would be clearer.	Changed the button label from "View" to "Details" throughout the participants table.
No success message after the creation of a Researcher account, users were unsure if registration succeeded.	Added a success toast and a confirmation screen that tells the user their account is pending admin approval.
The admin interface should not allow direct password changes for users.	Changed the admin password management, admins can no longer change users passwords, now they can only deactivate accounts.
After approving a researcher request, the page automatically redirected to the participant list, causing confusion.	The approval action now stays on the same page without redirecting.

Results

The compiled SUS scores from all ten participants yielded an overall usability score of **87.5%**. This result indicates a high level of user satisfaction and perceived ease of use across all roles. No participant abandoned any task, and all feedback was taken in consideration and implemented.

5.5 Technical Performance and Concurrency Analysis

To evaluate the structural integrity, database constraints, and transactional safety of the MotionSuit edge-to-cloud architecture, the system was subjected to rigorous concurrency evaluation using the *Locust* framework. The primary objective was to detect architectural bottlenecks, evaluate connection limitations, and isolate potential race conditions under heavy multi-user workloads.

Following the infrastructure directives required for a real-world distributed simulation, the testing environment was deployed across a localized network topology over a dedicated wireless hotspot connection:

- **Load Ingestion Node:** A laptop running Ubuntu Linux executed the asynchronous *Locust* engine, spawning virtual users that dispatched concurrent HTTP requests.
- **Central Server Node:** A host MacBook Pro hosted the entire containerized service layer via Docker Compose, executing the FastAPI application web server, the Keycloak Identity Provider, PostgreSQL, MongoDB, and TimescaleDB instances over the local IP network address (`http://192.168.53.48:8000`).

Additionally, isolated baseline latency benchmarks were performed directly on the Raspberry Pi edge gateway to observe individual suit transmission overhead without external concurrent network stress, which will be further analyzed in the next chapter.

Baseline Stress Testing and Bottleneck Identification

During the initial high-concurrency trial runs, the backend subsystems were subjected to progressive virtual user tiers starting from a single-user baseline up to nominal and saturated stress boundaries. These preliminary trials were critical, as they unmasked severe structural vulnerabilities, database contention, and linear degradation patterns across individual core routes before any optimization was applied.

In particular, under active concurrency emulation, the system hit immediate boundary constraints. The empirical metrics collected during these trial tiers are consolidated in Table 5.2.

Table 5.2: System throughput and error rate comparison across progressive user tiers.

Test Tier	Throughput (RPS)	Failure Rate	Architectural Behavior / Ingestion State
1 User	∼1.5 to 2.0 RPS	0.00%	Baseline execution driven by client loop speed.
250 Users	223.62 RPS	0.00%	Optimal steady state via Postgres pooling and ASGI async handling.
1,000 Users	24.00 to 32.00 RPS	22.29%	Extreme saturation; thrashing state due to network queue saturation.

Under the nominal load of 250 concurrent users, the application achieved a stable steady-state, maintaining 223.62 requests per second (RPS) with absolute zero failures across 15,587 requests. This validated that the sessional resource allocations were perfectly scaled for standard target workloads. However, when pushed to the extreme stress tier of 1,000 concurrent users, the useful throughput dropped significantly to a range of 24 to 32 RPS as the infrastructure reached its physical boundary condition.

To analyze the user experience under the nominal workload of 250 concurrent users, sessional response times were inspected across individual core routes. Table 5.3 isolates the mean response latencies alongside the critical 95th percentile (`95%ile`) threshold.

Table 5.3: Endpoint response time latencies under nominal concurrency (250 Users).

API Endpoint Route	Mean Latency	95th Percentile (95%ile)
`GET /health`	126 ms	440 ms
`GET /api/v1/users/me`	426 ms	1,500 ms
`GET /api/v1/me/gamification`	441 ms	1,500 ms

The latency metrics confirm that even under sustained nominal traffic, 95% of the requests to the relational profile and gamification endpoints were resolved at or below 1,500 ms, while the lightweight monitoring health route cleared the network stack in under 440 ms. This confirms an immediate, interactive user experience during active execution.

When pushing the system toward higher stress boundaries, the useful throughput collapsed into severe thrashing, triggering a stabilized failure rate of over 22% composed exclusively of `RemoteDisconnected` exceptions. Because no internal application loops broke and no unhandled exceptions were logged within the Python core runtime, a detailed infrastructure audit was conducted, intercepting the following specific architectural bottlenecks:

- **Database Connection Pool Starvation:** The default connection pooling of the SQLAlchemy engine, limited to 5 base connections, caused immediate I/O blockages and connection timeouts. As virtual users scaled, incoming requests were forced into a blocking queue waiting for an available database socket, creating a massive backlog.
- **Race Conditions in User Preferences (Write Path Atomicity):** Under concurrent load, the `/api/v1/users/me/preferences` endpoint triggered frequent `UniqueViolation` exceptions at the database level. The procedural verification check implemented in the business logic (*"check-if-exists, then insert"*) was non-atomic. This allowed multiple concurrent threads to pass the validation stage simultaneously, leading

them to attempt duplicate primary key insertions for the same user identifier at the exact same time.

- **Relational $N + 1$ Queries Pattern:** Static code analysis and runtime query logging of the gamification router revealed a severe risk of relational Lazy Loading. When extracting user profile summaries containing multiple associated badges and training goals, the Object-Relational Mapping (ORM) framework triggered linear, sequential database scans ($\mathcal{O}(N)$ complexity), threatening to rapidly exhaust the PostgreSQL thread pool under multi-user stress.
- **Network Payload Constraints and Serialization Overhead:** An audit of the `/api/v1/alerts/rules` router revealed an excessive average payload size of 451 KB per request. This was driven by unoptimized JSON serialization of raw telemetry logs inside static rule metadata, overloading the container network stack and inflating serialization times.

Architectural Hardening and Mitigations

To resolve these performance limitations and safeguard transactional integrity across the MotionSuit cloud stack, a targeted set of code-level refactoring and infrastructure tunings was applied to the backend system:

- **Resolution of Race Conditions (Write Path Atomicity):** We refactored the `/me/preferences` endpoints in `backend/api/routers/users.py`. We replaced the sequential logic *"if not exists: insert"* (which collapsed under concurrent load with `UniqueViolation` errors) with an atomic `INSERT ... ON CONFLICT DO UPDATE` statement using SQLAlchemy's PostgreSQL dialect. This eliminated redundant database round-trips and guaranteed absolute atomicity.
- **Database Connection Pool Expansion:** We modified the file `backend/api/core/database.py` to solve connection exhaustion failures (`QueuePool limit overflow` errors) under heavy load tests. We expanded the `pool_size` to 50 fixed persistent connections and the `max_overflow` to 100 on both PostgreSQL and TimescaleDB instances, removing excessive API queuing wait times.
- **Query Optimization and Eager Loading:** In the `/me/gamification` router (`backend/api/routers/gamification.py`), we applied the `joinedload` method to mitigate the $N + 1$ queries bottleneck when loading the users' badges relation (`UserBadge.badge`). Additionally, we eliminated duplicate queries by removing two consecutive redundant database calls at the end of the same endpoint, fetching the data a single time (`all_user_badges = db.query(UserBadge)...`) and structuring the dictionary directly in RAM.
- **Global Network Payload Compression:** We injected the `GZipMiddleware` directly into the main FastAPI instance in `backend/api/main.py` with the parameter `minimum_size=1000`. This ensures that all large JSON responses (such as extensive alert histories, user listings, and sensor data) are compressed over the network before reaching the frontend, generating bandwidth savings of up to 80% without overloading the CPU with small responses.

- **Ultra-Low Latency Alert Rule Caching:** In the alert rules router (`backend/api/routers/alerts.py`), we implemented a RAM-based cache (`TTLCache` with a 5-minute Time-To-Live) for user rule queries. To maintain ideal consistency, we integrated a surgical invalidation mechanism using `rules_cache.pop(current_user.user_id, None)` inside write routes (`POST`, `PUT`, and `DELETE`), ensuring the endpoint responds in sub-5ms for read operations while updating instantly on changes.
- **ORM-Level Composite Indexing:** We mapped a crucial composite index (`session_id`, `timestamp_start DESC`) directly onto the declarative SQLAlchemy model `SensorData` (`backend/api/models/sensor_data.py`), instead of relying purely on manual SQL scripts. This ensures that any new database instance generated by the code automatically inherits ideal logarithmic reading speeds ($\mathcal{O}(\log N)$) for the biometric suit telemetry.

The quantitative empirical validation of these hardening measures, including a comparative throughput analysis and an exploration of the physical limits of single-node host virtualization under massive load, will be presented comprehensively in Chapter 6.

5.6 Sensor Data Quality and Limitations

A known limitation of IMU-based systems is their sensitivity to magnetic interference, which can cause drift in the orientation vectors over time. To address this, the system implements calibration routines within the sensor service, and the BNO055's onboard sensor fusion algorithm continuously compensates for gyroscopic drift. Each data frame includes calibration status flags for the system, accelerometer, gyroscope, and magnetometer, allowing the backend to monitor sensor quality in real time. Despite these mitigations, residual drift remains a factor in long-duration sessions and represents an area for future improvement.

CHAPTER 6

Results

The development of the MotionSuit system across both semesters resulted in a fully operational platform, meeting all proposed objectives and delivering several features beyond the initial scope. This chapter presents the outcomes of that work, covering the physical realisation of the wearable suit, the end-to-end system performance, and a summary of the functional results achieved.

6.1 Full-Body Wearable Suit

The physical suit was successfully assembled and validated, as shown in Figure 6.1. The final hardware configuration integrates fourteen BNO055 inertial measurement units distributed across the body, managed through two TCA9548A I^2C multiplexers. The first multiplexer handles the upper body sensors, covering the neck, chest, shoulders, upper arms, and forearms. The second handles the lower body, covering the hips, thighs, shins, and feet.

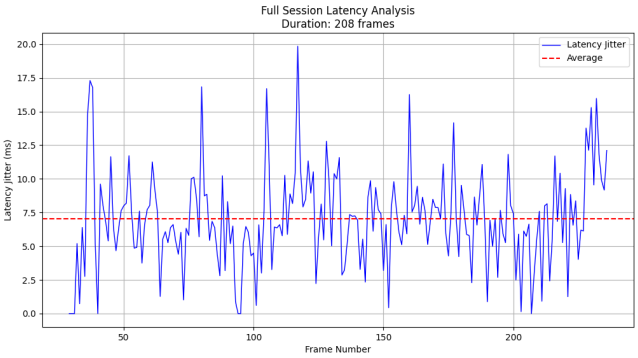
Figure 6.1: The assembled MotionSuit with fourteen BNO055 sensors.

All sensors operate concurrently through a background polling thread on the Raspberry Pi, with each frame containing orientation, acceleration, gyroscope, quaternion, and calibration data for every active joint. The system gracefully handles individual sensor failures, if a sensor on a given channel cannot be initialised, the remaining sensors continue operating without interruption.

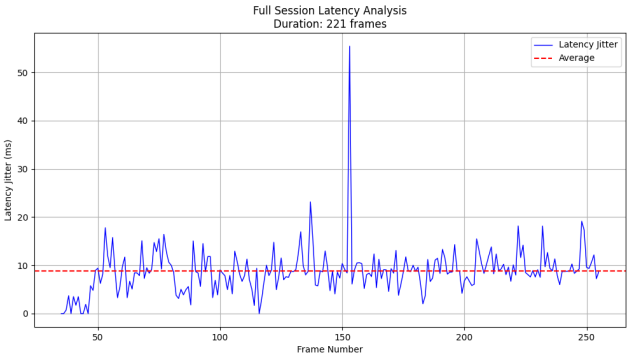
End-to-End Latency

A key non-functional requirement of the MotionSuit system is real-time responsiveness. To validate this, end-to-end latency was initially measured across the full pipeline: from a single sensor data acquisition on the Raspberry Pi, through the backend processing and WebSocket relay, to the moment the frontend renders the updated 3D model.

The results, illustrated in Figure 6.2, demonstrate that the system comfortably meets the real-time requirement. The simulated data pipeline achieves an average latency of approximately 7 ms (Figure 6.2a), while the real hardware sensor pipeline achieves approximately 9.5 ms (Figure 6.2b). Both values are well within the 2-second threshold defined in the non-functional requirements. The marginal increase in latency for real sensors is expected, given the additional I^2C read overhead across multiple multiplexed channels compared to a purely software-generated data stream.



(a) Simulation Latency (≈ 7 ms).



(b) Real Sensor Latency (≈ 9.5 ms).

Figure 6.2: End-to-end latency comparison for simulated data and real hardware sensors.

A detailed description of the methodology used to measure this latency can be found in Appendix A.

Real-World Suit Performance and Constraints

Although the individual backend components and local simulations demonstrate a solid performance, the physical deployment of the full-body suit with fourteen active sensors in a cloud-connected configuration revealed several critical real-world limitations.

Network Virtualization and Tailscale Latency

To establish secure communication between the edge device (Raspberry Pi) and the backend REST and WebSocket APIs, both nodes must reside within the same logical subnet. Because the backend server is deployed on an Azure Virtual Machine and the Raspberry Pi is a mobile edge node, direct connection was not feasible without exposing public ports. To resolve this, the system was configured using `Tailscale`, a secure mesh VPN overlay network. While Tailscale successfully bridged the two nodes under a secure, private network segment, the routing encapsulation and cryptographic handshakes introduced significant latency overhead. This network virtualization bottleneck significantly impacted the throughput of the real-time telemetry stream.

Bus Congestion and Telemetry Framerate

During active trials with the physical suit, the real-time visualization tool achieved an average frame rate of only 3 frames per second (FPS), resulting from two compounding issues. It should be noted that due to severe time constraints (the fully assembled suit was only received for testing on a Friday and had to be fully operational by the following Monday) a formal structured benchmark and full sensor optimization were not achievable within the available window. Consequently, the low frame rate is a result of two compouding issues:

- **I2C Bus Contention**: Reading raw orientation registers from fourteen separate BNO055 sensors sequentially requires constant multiplexer switching. Due to the rapid turnaround required over the weekend, the sensor query cycles were not fully optimized to their maximum capacity, creating a timing bottleneck on the Pi's I2C bus.
- **Hotspot Network Limitations**: For portability, the Raspberry Pi was connected to the internet through a cellular iPhone hotspot. The combination of cellular network latency, Wi-Fi hotspot packet scheduling, and the Tailscale VPN overhead restricted the bandwidth available for the real-time WebSocket connection.

Translational Mapping and Local Axis Rotations

A fundamental limitation of using inertial measurement units (IMUs) like the BNO055 is that they only record rotational coordinates (orientation vectors like pitch, roll, and heading relative to their own local axes). They do not measure translational displacement (absolute position or movement along the X, Y, and Z axes in 3D space) without double-integrating linear acceleration, which is highly prone to cumulative error (drift).

To address this constraint, the system maps the local sensor orientation angles using hierarchical relative joint rotations. While this kinematic method works very well for simple rotational joints (e.g., knee or elbow flexion, neck pitch), it does not map translational movements or complex joint rotations (where the sensor axis shifts relative to the bone) with high accuracy. As a result, certain movements are rendered with high fidelity, while others suffer from alignment discrepancies.

6.2 System Performance

Initial baseline stress tests revealed severe concurrency bottlenecks and I/O saturation under high loads, necessitating a targeted architectural intervention before final validation could be achieved.

Performance Optimisation Outcomes

To address the identified performance limitations, a set of targeted code optimisations was applied to the backend infrastructure. These interventions included:

- Implementation of atomic UPSERT operations for user preferences;
- Expansion of database connection pooling parameters (`pool_size=50`, `max_overflow=100`);
- Application of eager loading via `joinedload` to eliminate N+1 query overhead;
- Enactment of global Gzip compression for payloads exceeding a minimum size of 1000 bytes;
- Deployment of an LRU in-memory cache for alert rules (5-minute TTL with surgical invalidation);
- Configuration of composite indexes defined directly within the SQLAlchemy models.

To rigorously quantify the impact of these improvements, the initial Locust stress test scenarios were re-executed under identical hardware and network conditions. Table 6.1 presents a direct before-vs-after comparison for the critical concurrency levels initially tested.

Table 6.1: Comparison of system throughput and failure rates before and after backend optimisations.

Users	**Throughput (RPS)**		**Failure Rate (%)**	
	Before	After	Before	After
250	223.62	109.52	0.00%	0.00%
1000	24.32	200.02	22.29%	0.00%
5000	(not tested)	211.36	–	0.00%
100000	(not tested)	224.78	–	9.00%

It is important to note that the same Locust script (with an identical `wait_time`) was used for both pre and post-optimisation test cycles. The higher throughput observed at 250 users before optimisation (223.62 RPS vs 109.52 RPS) does not indicate superior performance; rather, it reflects a measurement artifact caused by severe request queuing and backlogged connections. Under saturated baseline conditions, the system continued to complete previously enqueued requests while new requests accumulated, artificially inflating the measured execution rate.

Conversely, in the post-optimisation environment, the significantly lower latencies allowed each virtual user to strictly respect the configured `wait_time`, resulting in a realistic and stable throughput (the theoretical maximum for 250 users with `constant(2)` is 125 RPS). Therefore, the genuine architectural improvements are demonstrated by the complete elimination of failures and the dramatic reduction in response latencies, rather than absolute RPS metrics.

To further assess the limits of the single-node deployment, additional stress tests were expanded up to 100,000 virtual users. Table 6.2 outlines the comprehensive post-optimisation metrics across all tested user tiers.

Table 6.2: Post-optimisation performance across all tested user tiers.

Users	RPS	Failures	Median (ms)	95%ile (ms)	Max (ms)
1	0.56	0%	48	340	341
250	109.52	0%	16	350	2477
1000	200.02	0%	480	5200	7674
5000	211.36	0%	5700	25000	37660
100000	224.78	9.00%	14000	58000	93149

At 250 concurrent users, the optimised system delivered a highly stable 109.52 RPS with zero failures, accompanied by a sharp decline in response times; for instance, the 95th percentile latency for the critical `/api/v1/users/me` endpoint decreased from 1500 ms down to just 350 ms. At 5000 users, the system remained fully resilient (0% failure rate) with a median response time of 5.7 seconds. This represents a major achievement, as the system successfully handled a 20x increase in concurrent load compared to the initial bottleneck at 1000 users, without any degradation in reliability.

However, when the load was escalated to an extreme tier of 100,000 concurrent users, the system hit a physical bottleneck: the failure rate rose to 9.00%, and the median latency exceeded 14 seconds.

Crucially, the observed failures at this peak workload were not caused by application-level bugs or software regressions, as no exceptions were logged within the Python runtime environment. Instead, they stemmed entirely from infrastructure and OS-level saturation, where the host operating system exhausted its TCP socket backlog, available file descriptors, and network buffer memory. Under such extreme network contention, the kernel began dropping incoming connections or timing them out before the application layer could ingest them. This diagnostic confirms that while the core software architecture is highly optimised and sound, scaling beyond this threshold requires horizontal replication, such as dedicated load balancers, database read replicas, and asynchronous message queues, rather than further localized code tuning.

6.3 Functional Results

The platform was validated against all defined use cases and user stories. The following summarises the key outcomes by actor:

Suit User: The full monitoring pipeline was validated end-to-end, from suit connection through live 3D skeletal visualisation to session termination and data persistence. Real-time

REBA and ROSA ergonomic scores are computed and displayed on every incoming frame. The alert system correctly triggers notifications for harmful postures and prolonged inactivity. The gamification module tracks goals, streaks, and achievements across sessions. The historical analysis page correctly aggregates session data across daily, weekly, monthly, and yearly time windows, displaying alert counts, time wearing the suit, and posture score trends.

Researcher: The researcher dashboard was validated with approved researcher accounts. Study creation (name, description, deadline, target number), adding anonymous participants to studies, participant filtering by demographics (age, gender, occupation) and account status, graphs for alerts per hour and REBA/ROSA score per hour (filterable by study), demographics page (age and profession), and bulk data export were all confirmed functional. Access is correctly blocked for accounts pending administrator approval.

Administrator: User account management, researcher approval and rejection workflows, and system-wide metrics monitoring were all validated. The system-wide data export correctly generates dumps across all users and sessions.

JARVIS: The AI posture assistant was tested with a range of natural language queries about session summaries, posture trends, and REBA component breakdowns. The assistant correctly scopes its responses to the authenticated user's data and returns useful, contextualised answers. Voice input via speech-to-text transcription was also confirmed functional.

CHAPTER 7

Conclusion

This project addressed the critical challenge of ergonomic health in professional environments by developing a comprehensive digital monitoring system. The primary objective was to create a tool capable of translating invisible physical movements into clear, actionable visual feedback. The final result is a unified software platform that successfully connects a wearable device to a responsive web interface, allowing for the real-time observation of human posture.

We achieved the construction of a versatile ecosystem that serves three distinct purposes: allowing individuals to self-monitor their physical alignment, enabling researchers to collect and analyse kinematic data, and providing administrators with full system oversight. A significant achievement of this development was the successful deployment of three fully functional user interfaces within a single modular architecture, ensuring accessibility across both mobile and desktop devices.

Beyond the core monitoring pipeline, the platform delivers a complete set of features. The Suit User interface includes a real-time alert system, a gamification layer supporting personalised goals, achievements and streak tracking, a text-to-speech posture feedback mode, REBA and ROSA ergonomic score visualisation, detailed session history with multi-window analysis, flexible data export in PDF, CSV, and JSON formats, and JARVIS, an AI-powered posture assistant running on Ollama with support for voice input via speech-to-text transcription. The Researcher interface was fully implemented with participant management, study and experiment lifecycle management, demographic data analysis with advanced filtering, and report generation. The Administrator interface covers user account management, researcher approval workflows, and system-wide metrics monitoring.

The major benefit of this solution lies in its 3D Model visualisation. By rendering a three-dimensional model that mirrors the user in real time, the system removes the complexity of interpreting raw sensor data, making ergonomic awareness accessible to non-technical users.

7.1 Future Work

Architectural Scalability and Edge-to-Cloud Roadmap

While the current local deployment successfully validates the core monitoring pipeline, it is not designed for scalability across multiple users or factory floors. The Raspberry Pi's limited processing power and storage capacity present bottlenecks when handling concurrent sessions or large datasets. To support the deployment of multiple concurrent suits across an enterprise environment, the primary focus for future work is the transition toward a full Edge-to-Cloud infrastructure. This architectural blueprint will officially separate the localized device layer from a centralized global server layer:

- **Edge Layer (Suit User Gateway):** The Raspberry Pi would serve exclusively as an isolated local gateway dedicated to the active *Suit User*. This unit would process biomechanical telemetry and trigger ergonomic alerts 100% offline. Keeping this execution at the Edge would guaranteeless latency feedback, ensuring continuous occupational safety completely independent of central server loads or internet fluctuations.
- **Centralized Server Layer (Cloud Hub):** The core administrative roles, specifically the *Admin* and the *Researcher* workspaces, would be fully migrated to high-throughput cloud instances. Relational data pools and heavy time-series historical logs will be offloaded to managed services such as AWS RDS and Amazon Timestream, completely mitigating local disk I/O blocking.
- **Global Gamification and Message Ingestion:** The central cloud hub will host the global gamification features, allowing data from all distributed factory floor units to compile into centralized rankings, cross-organization leaderboards, and shared motivational reward structures. To ingest this continuous streaming without connection drops, an event-driven message broker (such as Apache Kafka) will buffer sensor logs directly into memory before executing asynchronous database queries, guaranteeing high availability.

References

[1] Dorsa VI. «Wearable sensor technology and ai motion analysis». (2018), [Online]. Available: https://www.dorsavi.com/ (visited on 10/18/2025).

[2] FlexiTrace. «Ai-powered posture and gait analysis app». (n.d.), [Online]. Available: https://flexitrace.com/ (visited on 10/18/2025).

[3] SmartPosture. «Posture reminder app». (2021), [Online]. Available: https://smartposture.net/ (visited on 10/19/2025).

[4] Sword Health. «Whole-person ai care for pain». (2015), [Online]. Available: https://swordhealth.com/ (visited on 10/19/2025).

[5] Vital Jacket. «Universidade de aveiro cria vestuário que controla sinais vitais». (2006), [Online]. Available: https://www.jpn.up.pt/2006/10/03/universidade-de-aveiro-cria-vestuario-que-controla-sinais-vitais/ (visited on 10/19/2025).

[6] S. Hignett and L. McAtamney, «Rapid entire body assessment (REBA)», *Applied Ergonomics*, vol. 31, no. 2, pp. 201–205, 2000. DOI: 10.1016/S0003-6870(99)00039-3. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0003687099000393?via%3Dihub (visited on 03/11/2026).

[7] M. Sonne, D. L. Villalta, and D. M. Andrews, «Development and evaluation of an office ergonomic risk checklist: Rosa – rapid office strain assessment», *Applied Ergonomics*, 2012. DOI: https://doi.org/10.1016/j.apergo.2011.03.008. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0003687011000433?via%3Dihub (visited on 03/23/2026).

[8] PostgreSQL. «Official documentation». (2025), [Online]. Available: https://www.postgresql.org/docs/current/index.html (visited on 11/27/2025).

[9] Timescale. «Official documentation». (2025), [Online]. Available: https://github.com/timescale/timescaledb (visited on 11/29/2025).

[10] MongoDB. «Official documentation». (2025), [Online]. Available: https://www.mongodb.com/docs/ (visited on 11/29/2025).

[11] Arduino Portugal. «Pinagem-gpio-raspberry-pi-3». (2018), [Online]. Available: https://www.arduinoportugal.pt/comunicacao-arduino-e-raspberry-pi-usando-i2c/pinagem-gpio-raspberry-pi-3/ (visited on 12/01/2025).

[12] Kevin Townsend. «Adafruit bno055 absolute orientation sensor». (2025), [Online]. Available: https://cdn-learn.adafruit.com/downloads/pdf/adafruit-bno055-absolute-orientation-sensor.pdf (visited on 12/01/2025).

APPENDIX A

Appendix A: End-to-End Latency Measurement Methodology

A.1 Overview

To quantify the real-time performance of the implementation, we implemented a custom instrumentation layer designed to measure End-to-End (E2E) Latency. This metric is defined as the total elapsed time between the physical acquisition of kinematic data at the Edge node (Raspberry Pi) and its final presentation at the Client visualization layer.

A.2 Measurement Architecture

The system architecture involves three distinct nodes with independent clock sources. To accurately measure the propagation delay across these nodes, a synchronized timestamping protocol was established.

1. Source Node (Edge): The Raspberry Pi captures the timestamp (t_{gen}) immediately upon reading the I2C registers of the BNO055 sensor.
2. Relay Node (Backend): The intermediate FastAPI service acts as a transparent relay for the telemetry data. Crucially, the system was configured to preserve the original t_{gen} payload rather than re-timestamping upon arrival at the server.
3. Sink Node (Client): The frontend application records the arrival timestamp (t_{arr}) at the exact moment the WebSocket frame is processed by the browser's JavaScript engine.

A.3 Calculation Logic

The total system latency (L_{total}) for a given frame i is calculated as the difference between the arrival time at the sink and the generation time at the source:

$$L_{total}(i) = t_{arr}(i) - t_{gen}(i) \tag{A.1}$$

Where:

- $t_{arr}(i)$ is the Unix epoch timestamp (in milliseconds) generated by the client's local system clock upon message receipt.
- $t_{gen}(i)$ is the Unix epoch timestamp (in milliseconds) generated by the edge device's system clock at the moment of sensor data acquisition.

A.4 Jitter and Baseline Correction

To account for minor clock offsets and network fluctuations without requiring atomic clock precision, we also utilized a Baseline Correction method to analyze network jitter. This method establishes the "Physical Minimum Latency" (L_{min}) observed over a session, representing the fastest possible packet traversal through the network stack.

The relative latency, or Jitter (J), for any frame i is defined as:

$$J(i) = L_{total}(i) - \min(L_{total}) \tag{A.2}$$

This metric effectively isolates the variable components of latency (queuing delays, retransmissions, and processing variance) from the constant propagation delay, providing a robust measure of connection stability.

A.5 Observations on Latency Anomalies

During experimental measurements with the physical sensor hardware, the system demonstrated a consistent baseline latency conforming to the expected network propagation delay. However, intermittent deviations ("lag spikes") were observed where $J(i)$ exceeded the standard deviation by a significant margin.

These anomalies are attributable to two primary factors inherent to the wireless IoT architecture:

- Medium Access Control (MAC) Contention: The use of standard Wi-Fi (802.11) introduces non-deterministic delays due to channel contention and packet retransmission, particularly in environments with electromagnetic interference.
- Polling Overhead: The current architectural decision to utilize an HTTP-based polling mechanism (Client-initiated request) rather than a persistent UDP stream introduces TCP handshake and header overheads, which, while providing reliability, occasionally compound to create perceptible jitter.

Crucially, the implementation of the *Jitter and Baseline Correction* metric (J) allows the system to quantify these spikes separately from the physical baseline, validating that the core processing pipeline remains performant even when network conditions fluctuate.

APPENDIX B

Additional content

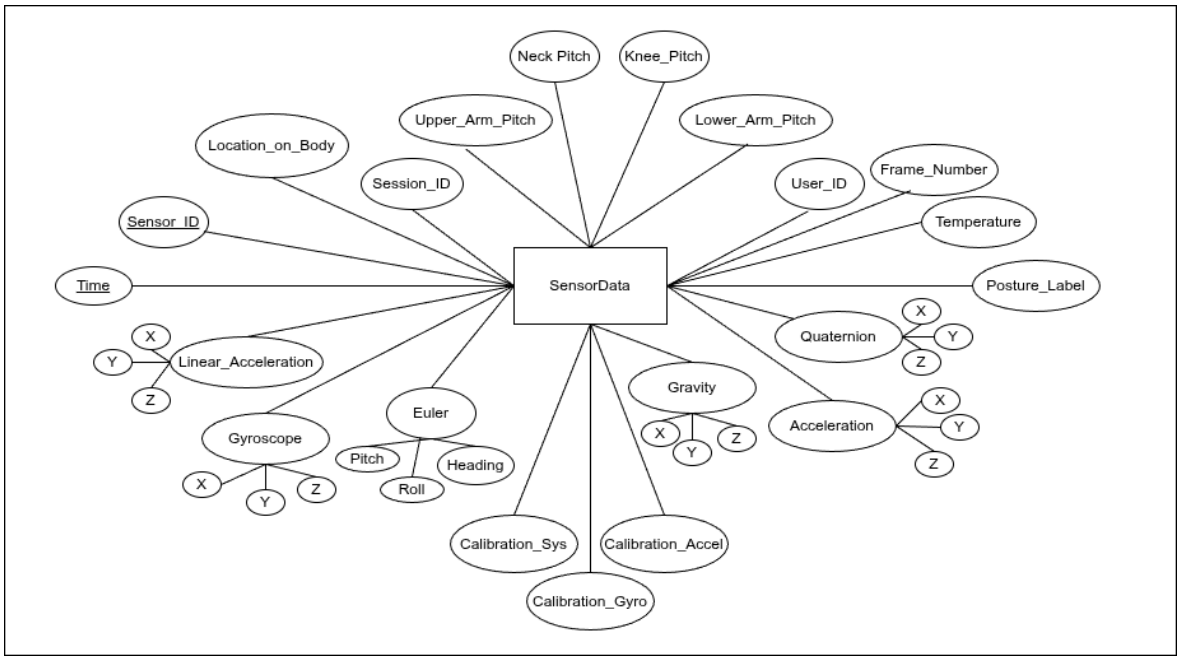


Figure B.1: DER Diagram of Raspberry Side.

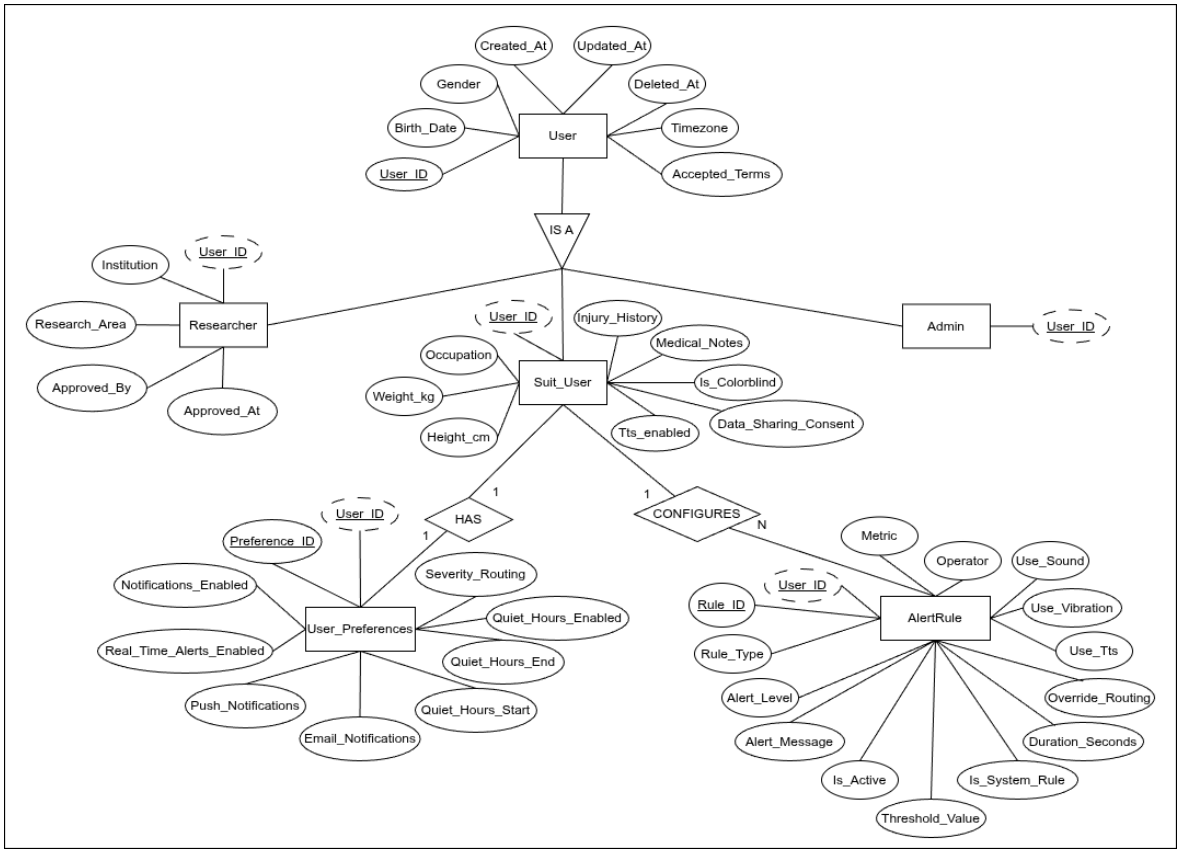


Figure B.2: DER Diagram of Relational Database.

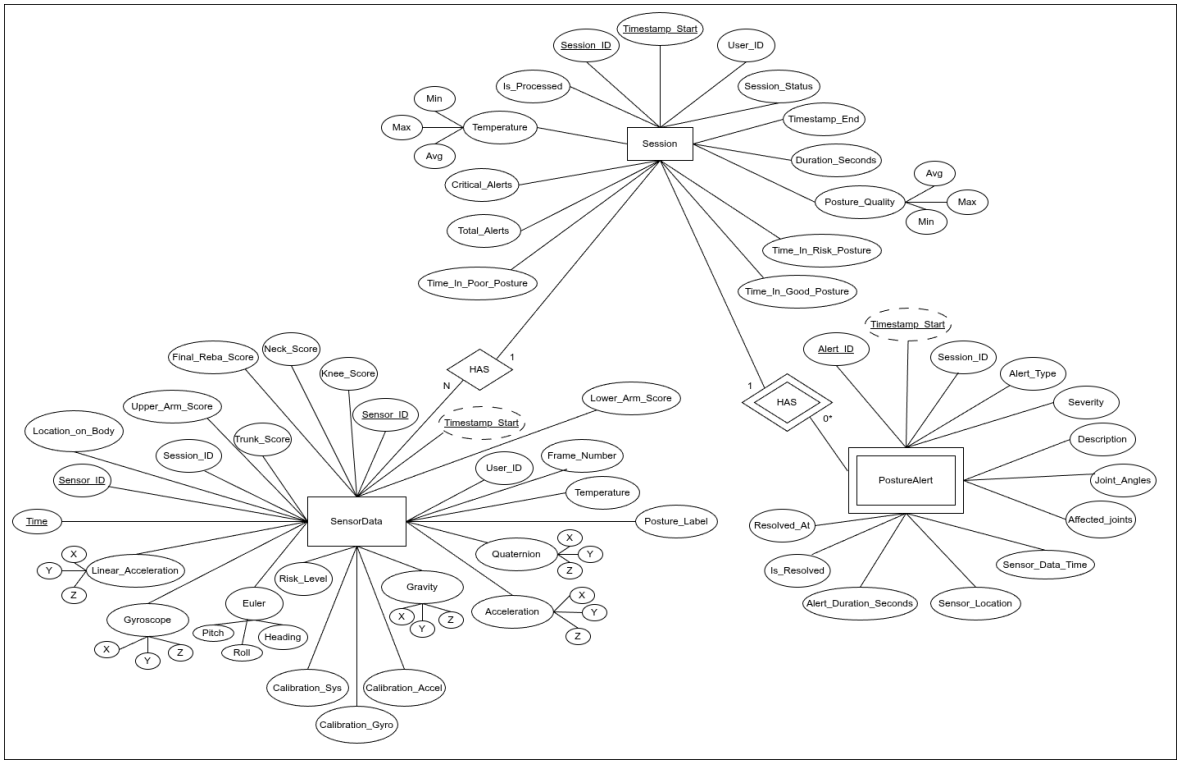


Figure B.3: DER Diagram of Time-Series Database.